

When Should Users Check? Modeling Confirmation Frequency in Multi-Step Agentic AI Tasks

Jieyu Zhou
Georgia Institute of Technology
Atlanta, Georgia, USA
jzhou625@gatech.edu

Aryan Roy
Georgia Institute of Technology
Atlanta, Georgia, USA
aroy389@gatech.edu

Sneh Gupta
Georgia Institute of Technology
Atlanta, Georgia, USA
sgupta852@gatech.edu

Daniel Weitekamp
Georgia Institute of Technology
Atlanta, Georgia, USA
dweitekamp3@gatech.edu

Christopher J. MacLellan
Georgia Institute of Technology
Atlanta, Georgia, USA
cmaclell@gatech.edu

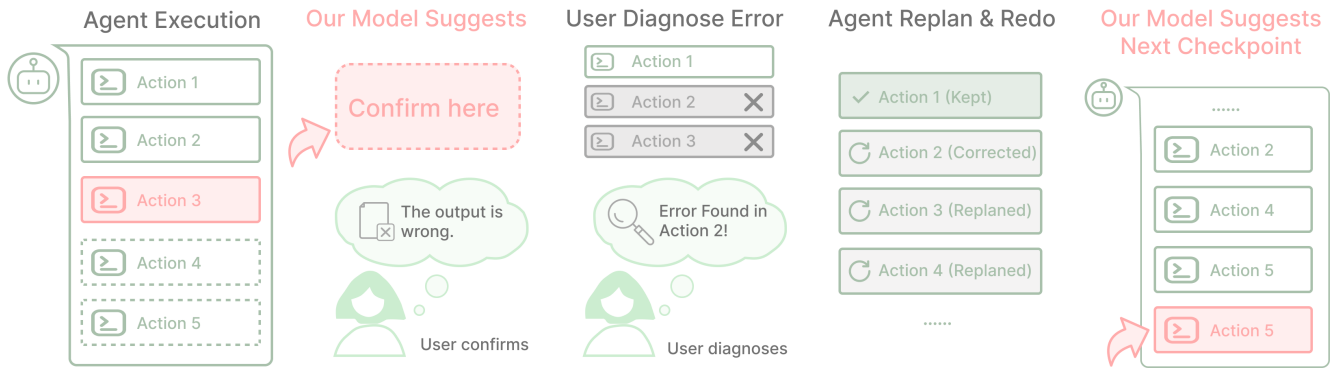


Figure 1: Confirmation-Diagnosis-Correction-Redo (CDCR) Pattern. Based on the expected time of user interaction under the CDCR pattern, our model decides whether the agent should proceed autonomously or prompt the user to step in and supervise.

Abstract

Existing AI agents typically execute multi-step tasks autonomously and only allow user confirmation at the end. During execution, users have little control, making the confirm-at-end approach brittle: a single error can cascade and force a complete restart. Confirming every step avoids such failures, but imposes tedious overhead. Balancing excessive interruptions against costly rollbacks remains an open challenge. We address this problem by modeling confirmation as a minimum time scheduling problem. We conducted a formative study with eight participants, which revealed a recurring *Confirmation-Diagnosis-Correction-Redo (CDCR)* pattern in how users monitor errors. Based on this pattern, we developed a decision-theoretic model to determine time-efficient confirmation point placement. We then evaluated our approach using a within-subjects study where 48 participants monitored AI agents and repaired their mistakes while executing tasks. Results show

that 81% of participants preferred our intermediate confirmation approach over the confirm-at-end approach used by existing systems, and task completion time was reduced by 13.54%.

ACM Reference Format:

Jieyu Zhou, Aryan Roy, Sneh Gupta, Daniel Weitekamp, and Christopher J. MacLellan. 2026. When Should Users Check? Modeling Confirmation Frequency in Multi-Step Agentic AI Tasks. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*, April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 20 pages. <https://doi.org/10.1145/3772318.3790655>

1 Introduction

With the advancement of Artificial Intelligence (AI), Agentic AI has emerged as a concept describing agents that assist humans in carrying out complex tasks [25, 36, 65, 76, 83, 94, 100, 106]. These AI agents have been deployed across diverse domains, including multi-modal devices [16, 75], web-based systems [37, 49, 51], and game controls [46, 92, 97]. Their rapid proliferation has reignited a long-standing HCI debate [34, 54, 59, 80]: should agents operate under direct human supervision and control or should they operate in a mostly autonomous fashion?

Most commercial AI agents today prioritize end-to-end autonomy, and rarely proactively ask users to confirm intermediate steps [6, 60, 64]. While some agents allow users to take control or interrupt execution [64, 72], these mechanisms are reactive: they are



This work is licensed under a Creative Commons Attribution 4.0 International License. *CHI '26, Barcelona, Spain*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2278-3/2026/04
<https://doi.org/10.1145/3772318.3790655>

only initiated when users halt execution upon noticing something is wrong, or when the agent halts to request missing information. Without proactive reminders, users often miss the window for timely intervention. As a result, errors frequently go unnoticed until the end of the trajectory. This is problematic because current agents remain highly error-prone, achieving only 30% accuracy on multi-step benchmarks [58, 103, 104], largely due to their limited capacity for generalizable reasoning and planning [81, 91]. Errors are especially costly in long-horizon agentic tasks, which often span 30 steps and 10–30 minutes of execution [14, 35]. A single mistake can cascade into complete task failure, forcing users to redo the entire process—doubling execution time and, through repeated API calls, adding both monetary costs [28, 43, 108] and increasing the agent’s carbon footprint [56, 67]. This motivates the need to determine when agents should proactively elicit user confirmation to prevent cascading failures.

The timing of such inspections is essentially a tradeoff between autonomy and control. At one extreme, too much user control (e.g., the user confirming after every step) prevents error propagation but imposes substantial interaction overhead, reducing efficiency to the level of manual execution or even worse. At the other extreme, too much automation (e.g., the user only confirming at the end) reduces the burden on the user but increases the risks of errors and costs associated with re-executing the tasks. The challenge is deciding when user checks should occur to balance the efficiency of automation with the safeguards of user control.

Prior HCI research has largely focused on **how** to provide user control, through visualizing the reasoning process, editing execution results, and providing error feedback [8, 9, 27, 28, 44, 45, 48, 52]. Some recent systems also consider **when** assistance should occur in user-led tasks, where the human is actively performing the activity and the agent decides when to offer help based on heuristic triggers or models of the user’s internal state [7, 50, 70, 71]. In contrast, we study the mirror case of agent-led workflows: an automated agent executes a multi-step task on the user’s behalf. The key question in this case is when the user should be brought back in to confirm. A notable exception is Horvitz’s seminal work on expected utility models for single-event decisions [34], but these approaches do not extend to long-horizon tasks, where sequential dependencies and cumulative costs shape overall outcomes.

In this paper, we address the open question of **when user checks should occur in long-horizon agentic tasks**. We first review representative agent benchmarks and deployed systems and present the results of a formative study we conducted with eight participants. The study revealed dissatisfaction with the confirm-at-end strategy used by existing agentic platforms. It also surfaced a recurring **Confirmation–Diagnosis–Correction–Redo (CDCR)** pattern employed by users to supervise agents. Building on these insights, we developed a decision-theoretic model that identifies periodic confirmation points that minimize total expected task completion time while ensuring task correctness. We directly validated our approach by comparing it to the confirm-at-end strategy using a within-subjects study with 48 participants under three domains. Results show that 81% of participants (39 of 48) preferred our intermediate confirmation strategy over the confirm-at-end baseline. We also found that our strategy reduced the average task completion

time by 13.54% compared to the baseline. We further discuss parameter realism, present simulations that demonstrate the model’s behavior under step-wise heterogeneity, and outline how our algorithm can be integrated to existing agent systems as a lightweight timing layer that triggers user-control hooks.

This paper makes four key contributions:

- 1) A formative study identifying user expectations for alternative confirmation strategies compared to the current confirm-at-end approach, and revealing a recurring CDCR pattern;
- 2) A decision-theoretic model for checkpoint placement that minimizes user interaction while ensuring correct task execution;
- 3) An empirical evaluation with 48 participants demonstrating both strong user preference (81%) for intermediate confirmation and a 13.54% reduction in task completion time relative to the confirm-at-end baseline;
- 4) A discussion framing our model not only as a confirmation scheduling tool but also as a design probe to surface potential directions for user-supervised systems.

2 Related Work

HCI research has primarily examined how to give users control, while reliability engineering has modeled when to intervene. We unify these perspectives by using mathematical models grounded in user behavior to determine when human control should occur in long-horizon agentic tasks.

2.1 Human Control in AI Agents

The tension between system autonomy and direct user control has been a recurring theme in HCI for decades [34, 54, 59, 80]. Current AI agents are highly automated, integrating context, tools, and memory to solve complex real-world tasks [87]. Yet these reasoning and tool-use systems are highly error-prone, yielding only 30% accuracy on 10-step tasks [26, 100]. Errors propagate like a snowball—growing exponentially and causing downstream failures [40, 110]. Agent self-correction is inconsistent and does not scale with task complexity [42]. These challenges highlight the necessity of human control.

Along this mixed-initiative spectrum, recent work has examined user-led tasks in which the human is the primary actor and the system decides when to offer assistance. Systems such as Satori, PrISM-Observer, ProMemAssist, and proactive programming assistants model users’ activity or cognitive state and trigger help at predicted good moments [7, 50, 70, 71]. For example, agents intervene when a user is likely to reach a particular step, forget an action, or benefit from a reminder or suggestion. In these systems, the core timing question is when an assistant should interrupt or scaffold an ongoing user task.

On the agent-led side, HCI researchers have proposed a wide range of tools for human involvement in domain-specific workflows, particularly in programming and data analysis [17, 22, 27, 28, 44, 46, 77, 82, 111, 113]. For instance, Cocoa builds on computational notebook interfaces to support co-planning and co-execution of research tasks [28], AGDebugger enables pausing and editing agent behaviors in programming [27], and Step-Phasewise decomposes complex analysis workflows for verification [44]. In addition,

research has expanded to general-purpose agents operating in open-ended environments. Systems like CowPilot [38] and Magentic-UI [61] have introduced mixed-initiative controls, enabling users to pause execution, reject plans, or approve specific steps. Similarly, Morae [69] and recent image creation agents [32] focus on detecting ambiguous decision points or maintaining explicit intent states to solicit user clarifications dynamically. These systems focus on how to give users control, but not when that control should be invoked in multi-step tasks. Our motivation is to provide a scheduling layer that triggers those existing hooks.

One useful way to study when interventions occur in agent-led workflows is through computational models of timing [19, 34, 88, 109]. For instance, Horvitz [34] proposed expected-utility models for deciding whether to invoke services, and Cockburn et al. [19] showed that users' probability-weighting biases shape when they engage or ignore assistive features. However, these works focus on single-event decisions and are not applicable to agentic AI, where tasks involve many interdependent steps. To capture the long-horizon nature of agent execution, we need to observe users' sequential behaviors around errors and build mathematical models that identify the optimal moments for user confirmation from a global, system-level perspective.

2.2 Error Prevention and Recovery Models

Outside HCI, reliability engineering offers inspiration. Researchers have long developed mathematical models to optimize inspection intervals for complex systems such as railway tracks, drainage infrastructure, and manufacturing plants [39, 85, 116]. These studies show how inspection strategies affect whole-system performance, considering dependencies such as workforce allocation, spare parts, and interdependent subsystems [23, 24, 93]. The central challenge is to balance preventive maintenance (early inspections to prevent failures) with corrective maintenance (recovering after failures occur), minimizing long-run costs by trading off inspection labor and detection probabilities against repair and downtime losses [10, 12, 84, 95]. For AI agents, the analogy is clear: deciding when to confirm execution steps likewise balances preventive checks with corrective recovery. But unlike engineering, the relevant cost is not purely economic—it is shaped by user experience, including confirmation burden and error recovery effort.

HCI work on errors has largely focused on how to prevent or repair mistakes. Examples include encouraging users to issue longer or repeated commands for accuracy [45], designing conversations that make error recovery more resilient [9], referring to third-party apps for additional information during repair [52], and studying whether error detection should be system- or user-initiated [48]. These studies enrich our understanding of error handling, but they do not provide a mathematical account of when intervention should occur across multi-step tasks.

In summary, while prior HCI work has richly explored how to give users control and how to prevent or repair errors, it has rarely asked when such intervention should occur across long-horizon tasks. Similarly, reliability engineering provides rigorous models of inspection timing, but they optimize purely for economic cost in physical systems. Our contribution is to bridge these two traditions: we introduce a mathematical perspective on the *when* question in

agentic AI, grounded not in labor or repair costs, but in the dynamics of user experience—balancing confirmation burden against recovery effort. This shift reframes human-agent interaction as a problem of scheduling when to switch initiative between agents acting and users verifying and correcting, opening new space for both theoretical modeling and practical design of adaptive confirmation strategies.

3 Grounding Confirmation Modeling in Agentic AI Contexts

Before building the model, we conducted a formative study to address two research questions:

- RQ1: How do users monitor AI agents during task execution? Is there any consistency in user behavior patterns across different task types and interface designs?
- RQ2: When should confirmation happen to support users in effectively monitoring AI agents?

To ensure that the observed user behaviors and the resulting modeling are grounded in realistic usage contexts, we first reviewed representative task benchmarks and existing AI agents (Section 3.1). Based on this review, we then designed a test environment that combines real-world tasks with deployed agentic systems to examine RQ1 and RQ2 (Section 3.2).

3.1 Agentic AI Task Types and Existing Systems

3.1.1 Task Types. To understand the landscape of agentic AI tasks and guide the design of our user study, we reviewed 12 benchmarks that cover a wide spectrum of real-world scenarios [16, 41, 58, 90, 96, 99, 102–105, 115]. These benchmarks are grounded in user-generated platforms such as StackOverflow, Reddit, and other public-facing systems, where users frequently ask how to complete complex or ambiguous tasks. Each benchmark contains around 300 pieces of task data. From these benchmarks, we identified four broad application domains—Office, Daily Life, Virtual Environment, and Mixed Workflow—each encompassing several representative task types (Figure 2, left). These domains provided the basis for selecting tasks in our formative study. Despite advances in LLMs, current models such as GPT-4 or Claude 3.5 still struggle to reliably complete long-horizon tasks. Reported success rates for benchmarks like OSWorld [103], SpreadsheetBench [58], and TheAgentCompany [104] often fall between 30% and 60%, with failure modes including incorrect tool use, misinterpreted instructions, or skipped steps. This relatively high error rate underscores the need for human-in-the-loop oversight, where users monitor and intervene during task execution.

3.1.2 Existing Systems. We tested tasks from the benchmarks mentioned above in nine agentic systems [2, 3, 6, 21, 29, 31, 60, 64, 89]. We also reviewed recent surveys on agent interface design [57, 101, 112]. Typically, these agents operate within remote desktop environments, where the agent perceives its environment through periodic screenshots. The vision-language model (VLM) reasons about the current state and user instructions, and then performs primitive actions, such as mouse clicks and keyboard inputs. For instance, when the agent perceives a hotel booking website, it will enter the destination city on that website automatically, as shown in

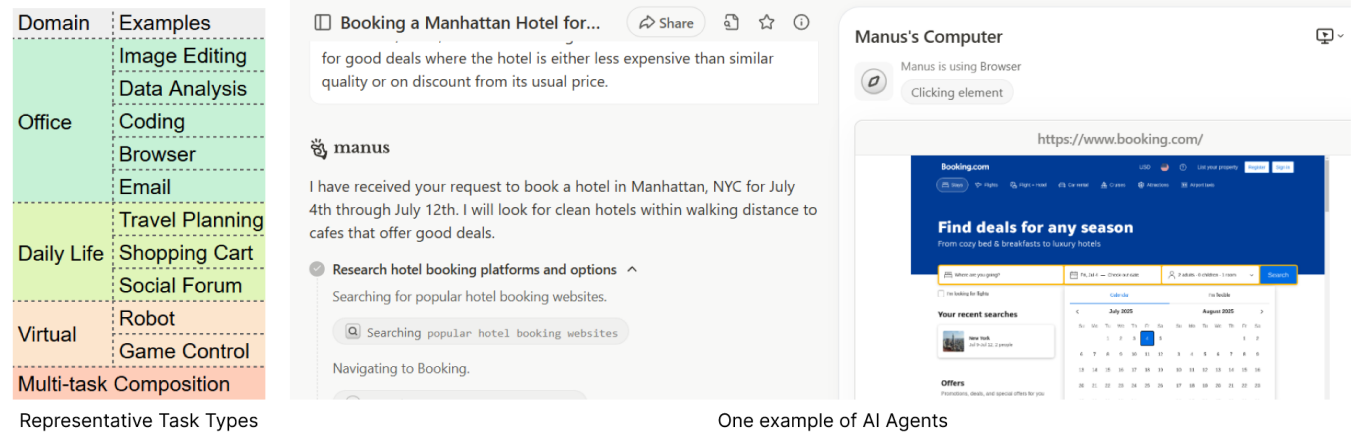


Figure 2: Representative Task Types and AI Agent Interface Example

the middle of Figure 2. Over multiple steps, the agent decomposes a high-level goal into a sequence of fine-grained interface operations.

From the interaction perspective, we observed three common UI patterns across existing agents: (1) in some agents, each step is a clickable node linked to its corresponding screenshot [2, 21, 29, 60]; (2) in others, the agent’s activity or reasoning process is visually embedded within the screenshot, with a sliding progress bar to navigate through the log [6, 31, 64]; and (3) some agents provide only screenshot playback without an activity log [3, 89]. While these UI patterns allow users to inspect what the agent has done, confirmation mechanisms are exceedingly rare, given the low accuracy rates of current agents and the high cost of errors. Most systems execute entire task sequences autonomously, while some can pause to let users take control [64, 72]. However, this pause mechanism is passive and reactive: the pause occurs only because the agent cannot proceed (e.g., missing payment credentials or login details), not because it seeks confirmation from users whether its execution so far is correct. As a result, reactive control provides no guidance on when users should check the agent’s progress, because it is entirely user-initiated. Consequently, most errors (e.g., clicking the wrong UI element or failing to load a page) are not detected in real time, and the agent either continues executing incorrectly or stops without recovery [42]. These gaps motivated us to explore how to schedule intermediate confirmations in ways that ensure timely user intervention without introducing unnecessary overhead.

3.2 Designing the Formative Study Environment

Drawing from prior benchmarks and real-world agent deployments, we selected representative tasks to cover a broad spectrum of domains. By using real-world tasks and existing systems, we aim to replicate users’ authentic experiences, ensuring that subsequent model development would be grounded in realistic usage contexts.

3.2.1 Environment Setup. We selected five tasks to represent a broad spectrum of domains, including everyday activities, office workflows, and virtual environments. To cover the three UI patterns

described above, we selected one representative agent for each pattern and randomly assigned tasks to these agents:

- Agent 1 Manus [60] (node-based navigation) — (1) Travel planning: booking hotels, flights, taxis, and restaurants; (2) Shopping cart: adding ingredients for making pizza to an online shopping cart.
- Agent 2 ChatGPT Agent [64] (embedded sequential navigation) — (3) Document processing: converting scanned invoices into a spreadsheet and emailing it to the reimbursement office; (4) Image editing: removing unrelated objects, changing backgrounds, and replacing objects in an image.
- Agent 3 Browser Use [62] (screenshot-only playback) — (5) Video game play: navigating through different terrains, boiling ingredients, and plating meals in the game Overcooked.

The Overcooked task runs in a custom game environment that requires direct programmatic control and access to internal state. In our setup, only Browser Use could reliably operate in this environment, so the Overcooked task was always paired with Agent 3. The remaining four tasks all use conventional web-based interfaces and are similar in complexity, so we randomly assigned them to Agent 1 and Agent 2 while ensuring that each handled one everyday task and one office workflow. This design gives us coverage of three distinct UI patterns while keeping task-agent pairing a nuisance factor rather than a primary variable of interest.

3.2.2 Procedure. We recruited eight participants for a 45-minute user study, structured in two phases. Each participant completed all five tasks, allowing us to compare how both task type and confirmation frequency influenced user behavior within the same individuals.

Phase 1 — RQ1 (User Behavior): In Task 1, participants were instructed to check every step to familiarize themselves with the agent. In Tasks 2–5, they confirmed results only at the end of task execution. Participants were asked to think aloud, verbalizing their judgments about correctness and describing how they located errors in the sequence of steps. **Phase 2 — RQ2 (Confirmation Preferences):** We conducted a semi-structured interview to explore participants’ desired confirmation frequency, comparing their experiences with

end-of-task versus step-by-step confirmation, and eliciting their ideal frequency.

All sessions were recorded and transcribed. We conducted a thematic analysis [13, 15], in which two researchers collaboratively coded the transcripts. We refined emerging themes through the lens of our research questions (RQ1 & 2).

4 Formative Study Findings

Our thematic analysis of the formative study data identified a dominant Confirmation–Diagnosis–Correction–Redo (CDCR) pattern where participants dealt with AI agent errors by reviewing the execution history from beginning to end. This strategy was evident across diverse task types and UI designs. We also observed some exceptions to this pattern (RQ1). For RQ2, participants expressed a desire for confirmation strategies beyond the prevalent “confirm-at-end” norm, suggesting opportunities for intermediate checkpoints. These insights informed the framing of our confirmation frequency model.

4.1 Confirm-at-end is not enough

Seven out of eight participants expressed their dissatisfaction with the current confirm-at-end strategy. As P4 noted:

“I did not like confirming at the end. Because the results were so far off from what I originally intended, that was a waste of time to do. Looking through the trace to try and find the error, I see that it was having a lot of other problems. If it had stopped there, maybe I could have clarified what my intentions were when I said the thing. Instead of wasting its time and getting something that was so far off from what I actually wanted to get. And if we could maybe confirm at least a little bit more frequently, I think that would really help my confidence in the agent, [and improve] my communication [with the agent].”

These findings suggest the dominant “confirm-at-the-end” approach should be reconsidered. Although Agent 2 [64] offers a reactive control mode (allowing users to seize control of the agent’s screen), six of our eight participants reported that reactive control still made them uncertain about when to check, failing to help them “spend less time monitoring” (P1). Current agent UIs left users in a passive “wait state” during execution, with no meaningful hooks to monitor or steer the run. Consequently, to catch errors in real time, users are forced to maintain constant vigilance; if they look away, errors are easily missed. In that case, users cannot disengage while the agent executes, which undermines the value of automation. As P6 noted, “If I have to check every step my agent is making, I’d rather do it myself.” These results highlight the need for a proactive, agent-initiated confirmation mechanism that reduces users’ monitoring burden. The central challenge is to strike a balance between the fixed time cost of reviewing every agent action and the expected time cost of needing to confirm task completion at the end and then possibly undertake a long post-hoc diagnostic and redo process if the agent failed. We discuss our model for balancing between these two extremes with checkpoint scheduling in the next section.

Scheduling confirmations requires consideration of two components: (1) the time costs for each part of the Confirmation–Diagnosis–Correction–Redo (CDCR) cycle, including the time associated with waiting for the agent to generate actions, and (2) the agent’s accuracy. We found that when accuracy was high, users were fine with less frequent confirmation.

4.2 Common Usage Patterns

4.2.1 Common Agent Errors. The most frequent agent error in our formative study was a mismatch between the activity described by the agent (i.e., the action listed) and what was actually executed (i.e., the screenshot). For example, in the travel planning task, the activity list indicated “click July 4th” as the start date, but the screenshot showed July 9th selected. Another frequent issue was that the agent did not always execute actions in full alignment with the user’s intended goal.

4.2.2 The CDCR Pattern. We identified a recurring Confirmation–Diagnosis–Correction–Redo (CDCR) sequence (Figure 1). After the agent completes a task, participants typically *confirm* that the final screenshot matches their expected result. If not, they first *diagnose* the error by scanning the entire execution history, often skimming the activity list or sliding through the screenshot recording. Once they get the whole picture of the execution process, they scan the execution sequence step-by-step from the beginning, reviewing the activity descriptions, reasoning traces, and associated screenshots along the way. After locating the erroneous step, participants usually *correct* the agent by telling it how to fix the mistake. In the final *redo* stage, participants wait for the agent to replan from the erroneous step to the end based on their correction.

4.2.3 Exceptions to the CDCR pattern. Across all 40 trials (5 tasks $\times$ 8 participants), 100% (40/40) of trials entered the Confirmation stage, 87.5% (35/40) proceeded to Diagnosis, and 82.5% (33/40) executed a Correction. Agent differences did not fundamentally change this behavior. However, variations in task type and user familiarity revealed several edge cases in how the later CDCR stages were instantiated. (1) Ambiguous confirmability: In 5 trials (2 hotel-booking and 3 Overcooked), participants failed to identify an existing error. In the hotel-booking domain, ambiguity in what counted as “wrong” made it difficult to diagnose issues: for example, the agent sometimes chose a hotel that was slightly more expensive but had better transportation access. While arguably suboptimal, this choice was not objectively wrong. In the three Overcooked trials, participants noticed the unexpected result and entered the diagnosis phase, but failed to identify the first error. Because the agent continued to act after the initial mistake, participants reported that they expected the agent to correct itself later, and therefore overlooked the original error location. (2) Non-linear diagnosis: Two users skipped intermediate steps during diagnosis in Overcooked. Because they are highly experienced with Overcooked, they jumped directly to key states: after noticing the player placing soup on the wrong pad. (3) Partial correction: In 2 document-processing trials, participants entered the confirmation and diagnosis stages but chose an alternative correction path. They preferred to download the result file and fix the error themselves rather than having the agent

re-execute the task, because the errors were minor and unlikely to affect subsequent planning.

4.3 Implications for Model Scope and Applicability

Edge cases in the previous subsection provide concrete examples of how users actually confirm, diagnose, and correct agent errors in practice. Building on these observations, we abstract a set of modeling assumptions that capture a common regime while making clear where the model does not apply.

- (1) The task has a clear distinction between correct and incorrect outputs.
- (2) The user is not deeply familiar with the task, and thus needs to review from the beginning to understand the agent's actions.
- (3) Errors can propagate, and without timely correction, they affect subsequent similar actions.
- (4) Once the current step is verified, the model treats all earlier steps as correct and only searches for errors in later steps.

While factors such as task structure and user preference are important for real-world deployment, our current modeling abstracts away these user-specific and task-structural influences to focus on the core trade-off between confirmation cost and error diagnosis cost. This simplification positions our model within a broader landscape of supervision patterns, while leaving these additional dimensions for further investigation.

Our intended application scenarios involve long, multi-step tasks with repeated sub-actions where each sub-action carries a non-trivial probability of error. For example, in image editing workflows involving repeated operations such as removing objects, changing backgrounds, and cropping images, each step is similar in structure but may still fail unpredictably. Our model aims to identify confirmation strategies that maximize efficiency in such contexts.

5 Modeling Confirmation as a Minimum Time Scheduling Problem

This section introduces a formal model of confirmation scheduling. Our goal is to determine *when* the system should prompt the user to confirm correctness during a multi-step agentic task so as to minimize the user time needed to execute a whole task correctly. In other words, at each step, our system decides whether the agent should proceed autonomously or whether the user should be asked to step in and supervise. We proceed in three stages: (i) we define the environment (states, actions, transitions); (ii) we derive the expected time of a single confirmation interval using the CDCR decomposition; and (iii) we present a dynamic programming algorithm to identify the minimum time checkpoints. Finally, we describe how our model's parameters can be personalized.

5.1 Problem Formulation

We can formulate the scheduling of confirmations as a stochastic decision process:

$$\mathcal{P} = (\mathcal{S}, \mathcal{A}, \mathcal{T}),$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, and $\mathcal{T}$ is the transition function.

5.1.1 State space $\mathcal{S}$. Given an agent action sequence of length N , there will be $2 \times (N + 1)$ states in the space. Among these, $2N$ states correspond to the points after each action has been executed, which we denote as $s_i = \text{correct}$ or $s_i = \text{incorrect}$ for $i \in 1, \dots, N$. The correctness value of s_i is determined by whether all actions up to step i have been executed correctly by the agent. For example, if the system is in state $s_3 = \text{correct}$, this means that the agent's first three actions have been executed correctly. In addition, there are two special states that are always correct: (1) an initial state s_0 , representing the point before any actions have been executed, and (2) a terminal state s_{done} , indicating successful completion of the agent's entire action sequence.

At any given point, it knows that all agent actions up to the last user-verified state are correct, but it does not know whether the states beyond this point are correct (see Figure 3).

5.1.2 Action space $\mathcal{A}$. In any given state s_i , the system can choose between one of two actions. It can either execute action $i + 1$ from the agent's plan (execute) or request confirmation from the user (verify). Note that the system does not choose the agent's action (e.g., button clicks, scrolling, tool use), but only whether to proceed autonomously with execution or seek user verification. In this way, the system determines whether the agent or the user takes the lead. When the system is in state s_0 it can only choose to execute because this state is always correct by default, so confirmation is unnecessary. Similarly, when the system is in the final state s_N , it can only verify to transition to the terminal s_{done} state because there are no more agent actions to execute.

5.1.3 Transition function $\mathcal{T}$. The transition function depends on the two action options chosen by the system. We distinguish two cases:

(1) *Execute*. When the system chooses to execute it will transition from one state to another according to the following transition probabilities:

- $p(s_i = \text{correct} | s_{i-1} = \text{correct}, \text{execute}) = p_{a_i}$
- $p(s_i = \text{incorrect} | s_{i-1} = \text{correct}, \text{execute}) = 1 - p_{a_i}$
- $p(s_i = \text{correct} | s_{i-1} = \text{incorrect}, \text{execute}) = 0$
- $p(s_i = \text{incorrect} | s_{i-1} = \text{incorrect}, \text{execute}) = 1$

where $1 < i \leq N$ and p_{a_i} corresponds to the probability that the agent will execute action i from its plan correctly. These transitions also capture the cumulative probability of errors; i.e., once an agent makes a mistake, all subsequent states will be incorrect. Note that there is no execute transition out of the s_N state. An important point here is that the correctness of all states after the last user-verified state is not directly observable to the system; it becomes known only when the user inspects them. To compute the probability that any given state s_i is correct, the system must weigh the transition from the previous state by its beliefs about the correctness of the prior state: $p(s_i = \text{correct}) = p(s_{i-1} = \text{correct})p(s_i = \text{correct} | s_{i-1} = \text{correct}, \text{execute})$. By chaining over transitions, we can compute the probability that any state j is correct given the last user-verified state i , where $i < j$: $p(s_j = \text{correct} | s_i = \text{correct}) = \prod_{m=i+1}^j p_{a_m}$. Note, we do not need to consider actions before the last user-verified step because we already know they are all correct.

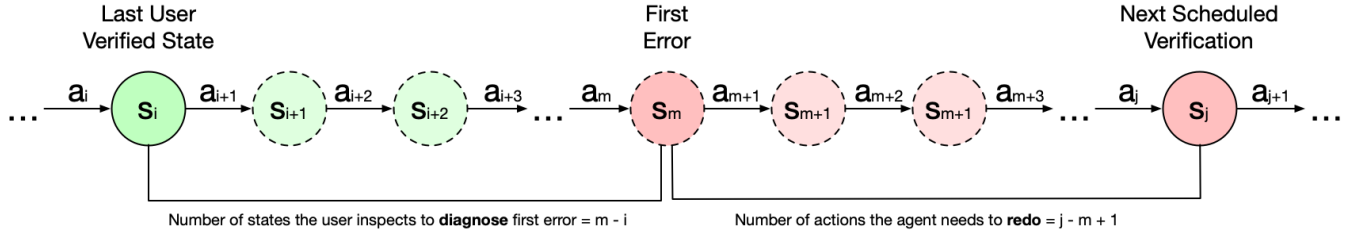


Figure 3: An example of the interval between the last verified state and the next confirmation point. The colors denote the correctness of the plan execution at that point, but this correctness is only observed when the user inspects them (denoted by solid circles). When the user finds an error in state s_j , they must go back to s_i and check each state one at a time until they find the first error. This requires them to inspect $m - 1$ states. Once the first error is identified, the agent then has to redo $j - m + 1$ actions. Note, the dashed circles signify that the correctness of a state is not known until diagnosis.

(2) *Verify*. When the system chooses verify, the user inspects the current state s_j and determines if it is correct.

- If s_j = correct and $j < N$, then $p(s_m = \text{correct})$ is set to 1 for all $m < j$ and system remains in s_j .
- If s_j = correct and $j = N$, the system transitions to the terminal state s_{done} .
- If s_j = incorrect, then the user goes back to the last verified state and starts inspecting each of the following states to diagnose where the first error state occurs. If the last state verified by the user was s_i = correct and during diagnosis the user determines that the first incorrect state is s_m = incorrect, then the system sets $p(s_k = \text{correct}) = 1$ for all $i < k < m$ and transitions to state s_{m-1} (e.g., see Figure 3). The probability that the first incorrect state will occur at state s_m follows a geometric distribution: $p(\text{first incorrect at } s_m | s_i = \text{correct}) = p(s_{i+1} = \text{correct}, \dots, s_{m-1} = \text{correct}, s_m = \text{incorrect}) = (\prod_{k=i+1}^{m-1} p_{a_k}) (1 - p_{a_m})$.

- t_{correct}^m : time for the user to specify what went wrong with action a_m and provide updated instructions to the agent to correct the mistake.
- t_{redo}^m : time for the agent to re-execute the action a_m after the correction.

If we know that the user has verified all the states up to s_i , then we can compute the expected time to go from i to N if the next checkpoint is at j , where $i < j \leq N$, as:

$$T[i, j] = t_{\text{confirm}}^j + p(s_j = \text{correct} | s_i = \text{correct}) \min_k T[j, k] + \sum_{m=i+1}^j p(\text{first incorrect at } s_m | s_i = \text{correct}) \left(\sum_{k=i+1}^m t_{\text{diagnose}}^k + t_{\text{correct}}^m + \sum_{k=m}^j t_{\text{redo}}^k + \min_k T[m-1, k] \right) \quad (1)$$

5.2 Time Cost

The system measures cost in terms of *user interaction time* (seconds), following the Confirmation–Diagnosis–Correction–Redo (CDCR) pattern, as shown in figure 4. We first derive the expected time cost from step i to its next confirmation point j . The interval begins at a state s_i = correct, which corresponds to the last state that the user has inspected and determined to be correct. The system then repeatedly chooses to execute actions until it reaches state s_j . At this point, the system chooses to ask the user to verify the correctness of s_j .

First, we denote the per-step time components in the CDCR pattern as follows:

- t_{confirm}^i : time for the user to confirm whether state s_j is correct. Concretely, the user compares the screenshot of the state with the agent’s activity list (i.e., the sequence of executed actions) to verify correctness.
- t_{diagnose}^m : the time required to inspect each intermediate state s_m by looking at its screenshot and the description of the single agent action executed immediately before.¹

¹This is different from t_{confirm}^i , which requires evaluating a sequence of agent actions rather than a single agent action.

The first term corresponds to the time it takes the user to confirm state s_j . The second term corresponds to the case where the state s_j is correct, which occurs with probability $p(s_j = \text{correct} | s_i = \text{correct})$. If this happens, then the expected remaining time is recursively defined as the expected time needed to traverse from j to N when the system chooses the next checkpoint k such that this time is minimized (denoted as $\min_k T[j, k]$). The third term corresponds to the cases where s_j = incorrect. If this happens, we sum over all possible locations of the first error, weighted by their probabilities. For each first incorrect state s_m , the expected remaining time is composed of four parts: the time needed to diagnose the error, the time needed to correct the mistake, the time needed to redo the incorrect steps, and the expected time needed to go from m to N if the system chooses the next checkpoint k such that this time is minimized.

5.3 Dynamic Programming Checkpoint Solver

Given the recurrent format of $T[i, j]$, we developed a dynamic programming algorithm to efficiently identify the best verification locations that minimize the total user time, as shown in Figure 5.

The algorithm starts at the end of the sequence, since there is always a verification at the end. It then works backwards, populating

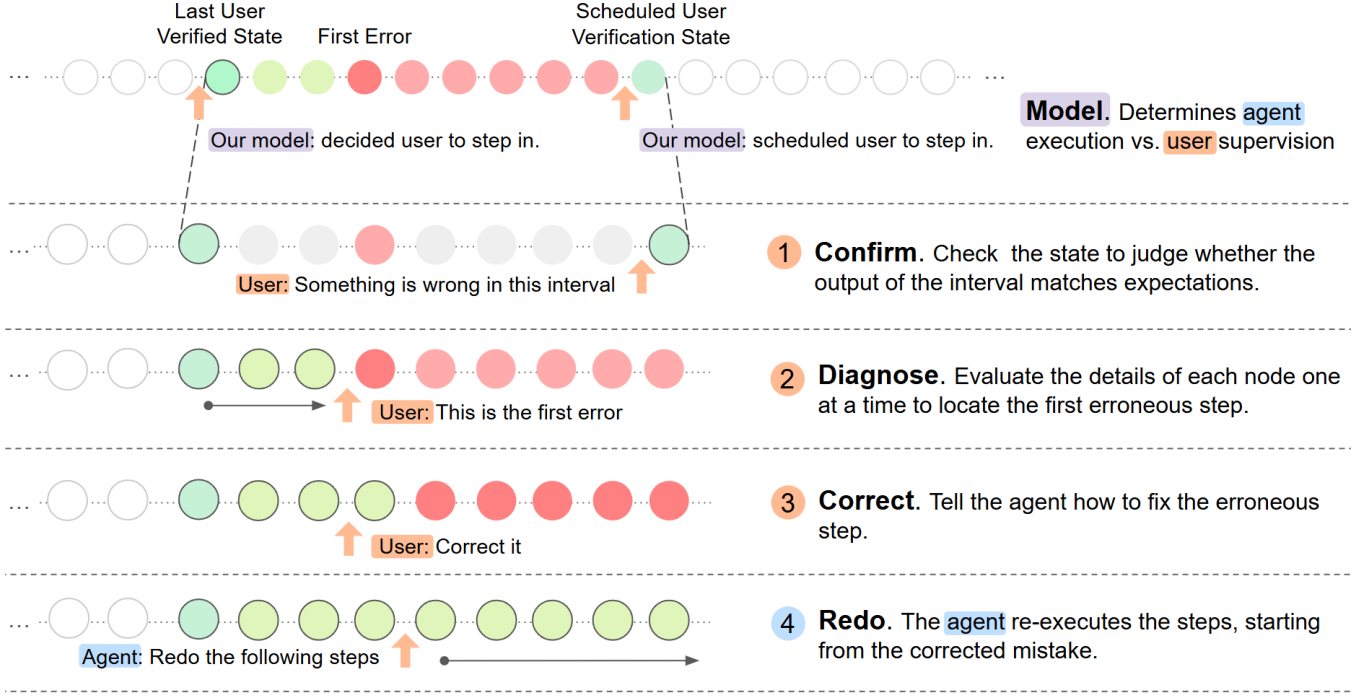


Figure 4: The Confirmation–Diagnosis–Correction–Redo (CDCR) interaction pattern and how our model schedules user verification. Given a multi-step agent plan, the model selects when the user should step in (purple labels). At each scheduled checkpoint, the user (1) Confirms whether the interval’s output matches expectations; if something looks wrong, the user (2) Diagnoses the interval by inspecting nodes one by one to locate the first erroneous step; the user then (3) Corrects the error; and finally the agent (4) Redoes the remaining steps from the corrected point.

the values of $T[:, :]$ according to the previously defined formula.² At each iteration, the algorithm computes the best next checkpoint for each starting location i , denoted as $next_ckpt[i]$.

The populated $next_ckpt$ vector can be used to determine the best checkpoint locations. For a given starting location i (which assumes $s_i = correct$), $next_ckpt[i]$ specifies the location of the best next checkpoint. For the example in Figure 5, when starting at state 1, the best next checkpoint is at state 2, if starting at state 2, the best next checkpoint is at 3, and finally, when starting at state 3, the next best checkpoint is at state 5.

If the user determines that a state is incorrect during one of the checkpoint verifications, then they follow the CDCR procedure. This might result in them ending up in an earlier state. If the agent action sequence does not change, then the precomputed T and $next_ckpt$ can be reused without computation. If, however, the action sequence changes, then these need to be recomputed.

5.4 Parameter Realism

In our system, we have both agent-side (probability and redo time) and human-side parameters (confirmation and diagnosis time). The

dynamic programming checkpoint solver itself is agnostic to how these parameters are obtained. Realistic values for these parameters can be calibrated from benchmark logs and user studies, and our algorithm permits these parameters to vary step-by-step, drift over time or update online, as illustrated via simulations (Appendix C).

5.4.1 Probability. On the agent side, evaluating agent performance is an active research area, with benchmarks reporting task accuracy [58, 90, 102–104, 115]. From such traces, we can estimate per-step success probabilities p_{a_i} . Crucially, the model does not require a single constant p across the entire task. We allow each step i to have its own success probability p_{a_i} , which can depend on the tool, page template, or subtask. Appendix C.1.1 shows how our model schedule checkpoints with heterogeneous step-wise accuracies.

For practical deployments, where general benchmark data may differ from the user’s specific environment, p_{a_i} can be dynamically estimated using a Beta–Bernoulli belief model: $p_{a_i} \sim \text{Beta}(\alpha_i, \beta_i)$. In this setup, accuracy metrics reported in benchmarks serve as informative priors. As environment conditions change (e.g., a website updates its layout or the user provides corrective feedback), the belief parameters can be updated online: incrementing α_i for autonomous successes and β_i for failures. The posterior mean would then serve as the input to our dynamic program. While our core algorithm is agnostic to the source of p_{a_i} , this Bayesian approach

²We drop $t_{correct}^m$ (the time needed for the user to instruct the agent on how to fix a_m) from the dynamic programming update because all errors need to be fixed no matter when verifications happen, so it turns out that dropping this term does not change the checkpoint locations.

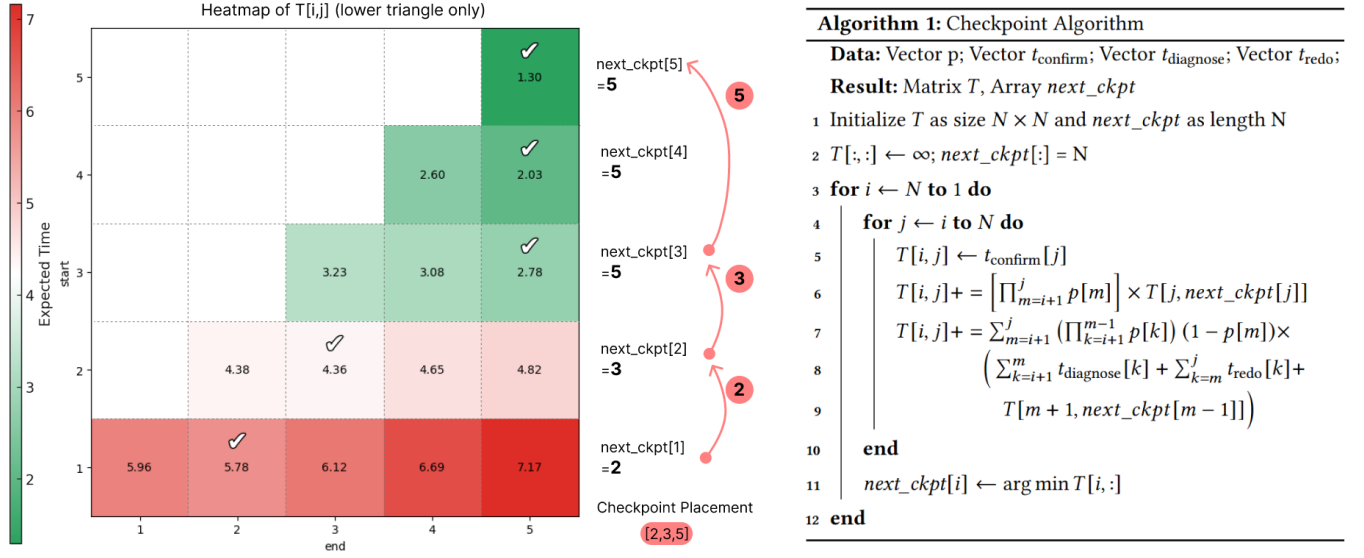


Figure 5: Dynamic programming algorithm and results for an agent action sequence of length $N = 5$ with action success probabilities $p = [0.7, 0.7, 0.9, 0.85, 0.85]$. Costs are parameterized as $t_{\text{confirm}}[k] = 1$, $t_{\text{diagnose}}[k] = 1$, and $t_{\text{redo}}[k] = 1$, for all $k \in \{0, \dots, N\}$. Each cell $[start, end]$ specifies the expected user time to correctly execute the rest of the action sequence from $start$ to N , if the system has a verification checkpoint at state end . White checkmarks indicate the $next_ckpt$ locations for each $start$. The right-hand annotations summarize the resulting checkpoint policy.

demonstrates how the system could adapt to user-specific reality. Determining the optimal weight of the prior relative to new observations remains a nuanced trade-off for future exploration.

5.4.2 Redo. The execution time t_{redo}^k can be grounded in timing benchmark results [55, 98, 107, 108]. Our model supports online replanning. When an error is confirmed at step m , we conceptually “cut” the original plan at that point: all steps up to $m - 1$ are now treated as confirmed, and the remaining steps form a residual task of length K' with a (possibly) updated action sequence. We then update the step-level parameters for this residual task (e.g., new success probabilities and redo times for the revised actions) and recompute the optimal checkpoint schedule for the remaining K' steps using the same algorithm. In other words, the same dynamic program is simply re-invoked on the current residual plan, yielding a receding-horizon policy that naturally accommodates updated plans, changing hazards, and revised redo times without altering the core recursion.

5.4.3 User Timing Parameter. On the user side, our model includes two timing parameters: t_{confirm}^k and t_{diagnose}^k . These are per-step quantities that may depend on interval-level or interface-level properties, such as the number of steps since the last checkpoint or the complexity of the UI (e.g., DOM size, density of interactive elements). Appendix C.1.2 presents one concrete example. In practice, these parameters can be treated as functions of such features and estimated from pilot data or interaction logs. While different factors influence the numerical values of the timing parameters—and therefore the resulting optimal schedule—they do not change the structure of our algorithm. A more detailed characterization of

how specific interface factors shape these two parameters is an important direction for future work.

These two parameters t_{confirm}^k and t_{diagnose}^k can also be personalized. These quantities can capture hierarchical effects, with individual reaction times treated as adjustments to the population average and refined as more user data becomes available. In this paper, however, we focus on the population-level model and leave personalization to future work.

6 Model Verification Study Design

We compared two confirmation formats: intermediate confirmation, guided by our minimum time scheduling model, and confirm-at-end, the baseline in current AI agents. The study tested whether intermediate confirmation improves efficiency and user experience.

6.1 Simulated Environment

Real AI agents vary widely in execution time and outcome quality. We built a simulated environment that fixed execution time and accuracy for each task, allowing us to isolate confirmation frequency as the primary factor. The user interface used a clickable-node tracking design (shown in Figure 6), which participants in our formative study reported as helpful for navigation and understanding task progress. We selected three representative task domains: shopping cart management, image editing, and the Overcooked game. These cover three broad categories: daily life, work-related, and virtual tasks, while also differing substantially in execution time, tool usage, and workflow complexity. Error cases were drawn from real agent failures observed in prior experiments: in shopping, the agent failed to replace an out-of-stock item; in image editing, the agent

Algorithm 1: Checkpoint Algorithm

Data: Vector p ; Vector t_{confirm} ; Vector t_{diagnose} ; Vector t_{redo} ;
Result: Matrix T , Array $next_ckpt$

- 1 Initialize T as size $N \times N$ and $next_ckpt$ as length N
- 2 $T[:, :] \leftarrow \infty$; $next_ckpt[:] = N$
- 3 **for** $i \leftarrow N$ **to** 1 **do**
- 4 **for** $j \leftarrow i$ **to** N **do**
- 5 $T[i, j] \leftarrow t_{\text{confirm}}[j]$
- 6 $T[i, j] += \left[\prod_{m=i+1}^j p[m] \right] \times T[j, next_ckpt[j]]$
- 7 $T[i, j] += \sum_{m=i+1}^j \left(\prod_{k=i+1}^{m-1} p[k] \right) (1 - p[m]) \times$
 $\quad \left(\sum_{k=i+1}^m t_{\text{diagnose}}[k] + \sum_{k=m}^j t_{\text{redo}}[k] + \right.$
 $\quad \left. T[m + 1, next_ckpt[m - 1]] \right)$
- 8 **end**
- 9 $next_ckpt[i] \leftarrow \arg \min T[i, :]$
- 10 **end**
- 11 $next_ckpt[i] \leftarrow \arg \min T[i, :]$
- 12 **end**

applied an incorrect mask that prevented object removal; and in Overcooked, the agent grounded to the wrong ingredient dispenser (e.g., selecting the tomato instead of the onion). Appendix B shows details about the simulation environment.

6.2 Within-Subjects Design

We employed a within-subjects design, where each participant completed two trials (intermediate and end) in all three domains. To control for potential biases, we introduced three layers of counterbalancing: task order, confirmation comparison order, and error location. 1) Task order. Each domain was treated as a block. We counterbalanced block order using a 3×6 Williams design, ensuring that each task domain appeared equally often in each position and that domain sequences were balanced across participants. 2) Confirmation strategy order. Within each block, the two confirmation formats for the same task were presented back-to-back. They were set to the same error location. To reduce familiarity effects, each trial followed the same structure but used different content (e.g., one shopping trial involved office supplies, another fitness items). Participants encountered different confirmation orders across the three blocks, and intermediate-first and end-first trials were balanced at the group level. 3) Error location. Error positions were sampled from early, middle, or late steps; shopping tasks included only early and late because these tasks were shorter. To prevent participants from anticipating that every trial would contain an error, we had each participant do one no-error trial. The no-error trial was implemented with fresh task content, assigned to one of the three task types, and counterbalanced across participants in both task type and confirmation format.

6.3 Baseline Setting

We selected confirm-at-the-end as our baseline because it is the dominant confirmation strategy in current agentic systems. As we observed in both system review and formative study in Section 3, current agents typically execute tasks autonomously from start to finish, and users only inspect the final result. Although some systems provide reactive controls, which are entirely user-initiated and offer no guidance on when the agent should proactively request confirmation. Because no deployed system offers such functionality, there is no established design to instantiate. In addition, introducing multiple hand-crafted policies would multiply study conditions without providing a principled comparison. For these reasons, we treat “confirm at the end” as the ecologically valid status quo. To complement the user study, we also provide simulations of several alternative confirmation strategies in Appendix D, allowing systematic comparison without burdening participants.

6.4 Procedure

The study began with a short tutorial explaining the agent background and confirmation methods. Each trial started with a task goal, and the Overcooked block included an additional introduction to the game rules. Each participant experienced 7 trials, including 2 (confirmation formats) $\times$ 3 (task types) + 1 no-error trial. After the trials, participants completed a post-study survey to capture participants’ reflections on the confirmation strategies. The survey included open-ended questions pertaining to perceived task success,

preferred confirmation style, and overall impressions of different confirmation strategies. It also included Likert-scale items assessing participants’ perceptions of intermediate checkpoints (e.g., whether they saved time, reduced burden, or disrupted flow).

6.5 Participants

We recruited 48 participants from a university community (18 female, 30 male). Regarding LLM usage, 41 participants (85%) reported using LLMs frequently (weekly or daily). In terms of familiarity with AI agents, 7 (15%) had never used them, 9 (18%) were beginners, 19 (39%) were intermediate users, and 13 (28%) were advanced users. Each session lasted about 15 minutes, and participants received \$5 compensation.

6.6 Variable Determination

The confirmation frequency in our model is determined by four parameters: t_{confirm} , t_{diagnose} , t_{redo} , and the accuracy rate. Although the model is fully capable of supporting heterogeneous, per-step values, in our study, we instantiated these parameters at the per-domain level, reflecting the relative uniformity of actions within each task domain. For instance, shopping consists of repeatedly adding different items to a cart based on a predefined shopping list (e.g., a recipe list or office-supply checklist); image editing involves common editing operations such as object removal or color adjustment; and Overcooked involves primitive moves such as navigating to objects or placing items. Using fixed per-domain parameters avoids the high-variance “lucky vs. unlucky” trajectories that arise when per-step probabilities and timings are randomly varied, which would inject noise unrelated to our research question and obscure differences between confirmation strategies. Thus, instead of mixing heterogeneous values within a single trial and producing unstable or uninterpretable trajectories, we evaluate three domains with distinct parameter profiles to show that our model operates robustly across diverse settings.

Parameter values for the three domains were grounded in pilot studies and prior work. (1) Confirmation and diagnosis times were estimated from a 10-participant pilot study with random checkpoints. In the main study, these averages were used as fixed inputs, with recalibration performed every 5–10 participants to prevent drift. We deliberately avoided per-user online adaptation, since stable personalization requires long-horizon usage data beyond the scope of this lab setting. (2) Execution time was estimated through formative agent runs and prior literature [55, 98, 107, 108]. Since steps are similar within each task, we applied fixed per-domain times: $t_{\text{redo}} = 20\text{s}$ for shopping (dominated by network and screenshot analysis), 10s for image editing (external tool calls), and 10s for Overcooked (agent reasoning plus game state processing). (3) For accuracy and steps, we set fixed per-step action correctness probabilities for each task domain, derived from formative agent runs: $p_{\text{shopping}} = 87.5\%$ with 8 steps, $p_{\text{image editing}} = 91\%$ with 12 steps, and $p_{\text{Overcooked}} = 93\%$ with 16 steps. These values were applied uniformly across all steps within a task. To contextualize this parameter regime, a per-step success rate of 0.9 across a 10-step task yields roughly $0.9^{10} \approx 0.35$ end-to-end accuracy, consistent with the $\sim 30\%$ accuracy of recent multi-step benchmarks [58, 103, 104].

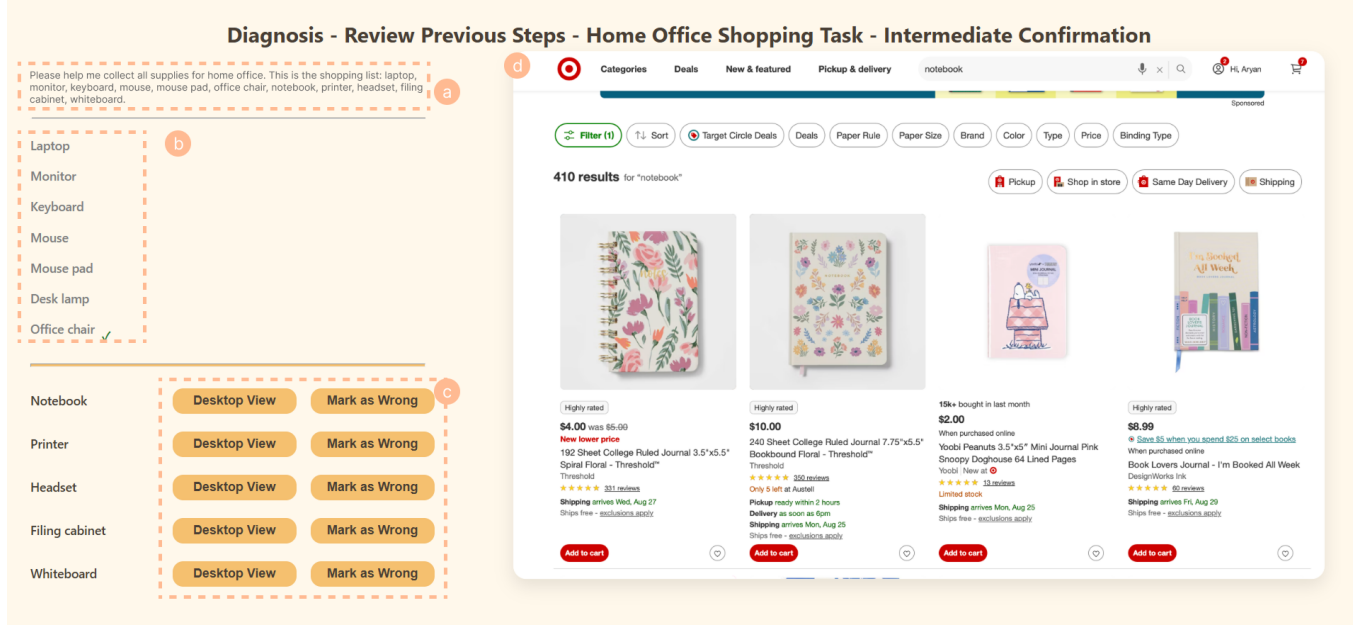


Figure 6: Diagnosis interface with clickable-node tracking for the Home Office Shopping task. (a) The user’s natural-language request to the agent. (b) Actions that the user has confirmed as correct. (c) The clickable-node tracking panel: each past action appears as a node that the user can click to open the corresponding agent execution state (“Desktop View”) or mark as the first wrong step (“Mark as Wrong”), which flags the error and triggers the agent to redo all subsequent actions. (d) The remote desktop view showing the agent’s execution state for the currently selected node.

7 Model Verification Results

In this section, we verify our model through both quantitative and qualitative analyses. While the quantitative results demonstrate what improvements intermediate confirmation achieves, the qualitative feedback explains why users value it and offers insights for future directions we discuss next.

7.1 Quantitative Results

Across all task domains, our model reduced the completion time of a full task trial (8–16 steps) by **13.54%**, corresponding to a savings of 35.84 seconds per task, compared to the baseline (End Confirmation) condition, $t(143) = 5.52, p < 0.001$. The baseline condition with end confirmation required an average of 264.70 seconds (95% CI [253.43, 275.98]), while intermediate confirmation using our model required 228.86 seconds (95% CI [219.73, 237.99]). Breaking down by task domain, the reductions were **17.44%** (47.60s) for Shopping ($t(47) = 5.25, p < 0.001$; our model mean = 225.33s, 95% CI [212.78, 237.88]), **7.46%** (17.41s) for Image Editing ($t(47) = 2.34, p = 0.024$; our model mean = 215.85s, 95% CI [203.42, 228.28]), and **15.64%** (49.03s) for Overcooked ($t(40) = 2.14, p = 0.044$; our model mean = 264.51s, 95% CI [236.71, 292.30]).

The effect of confirmation frequency varied by error location. When errors occurred early, intermediate confirmation substantially reduced total time ($\approx 29\%$). For mid-task errors, the benefit was minimal ($\approx 2\%$), while for late errors, intermediate checkpoints slightly increased time (+4.5%). This indicates that the value of

intermediate confirmation is greatest for early error detection and diminishes as errors occur later in the task.

Across all tasks, the average completion time was 246.78s (95% CI [236.58, 256.98]), consisting of 13% (32.08s, 95% CI [28.29, 32.95]) confirmation, 9% (21.85s, 95% CI [16.99, 26.74]) diagnosis, and 78% (195.55s, 95% CI [189.30, 201.79]) execution. Shifting from end to intermediate confirmation meant spending more time on confirmations, but this was offset by large savings in diagnosis (−17%) and redo (−22%). Moreover, the average time per confirmation decreased by 38%, as users only needed to consider the most recent steps rather than reasoning over the entire task history when verifying correctness. Intermediate confirmation reduced the mental tax of each checkpoint.

Beyond efficiency, reducing redo (−22%) also cuts repeated LLM calls, lowering both financial costs and the energy footprint of unnecessary computation [56, 68]. Although environmental impact is not our primary focus, this highlights an additional benefit: confirmation strategies that limit rework may also support more sustainable AI use.

7.2 Qualitative Findings

7.2.1 Overall preference. In our post-study survey, **81%** of participants (39 of 48) preferred the modeled intermediate confirmation strategy, compared to 15% (7 participants) who preferred end confirmation and 4% (2 participants) with no preference (Fig. 8, right). The Likert responses (Figure. 8, left) further support this preference: most participants agreed that intermediate confirmation reduced errors (81%), saved time (71%), lowered cognitive burden (63%), and

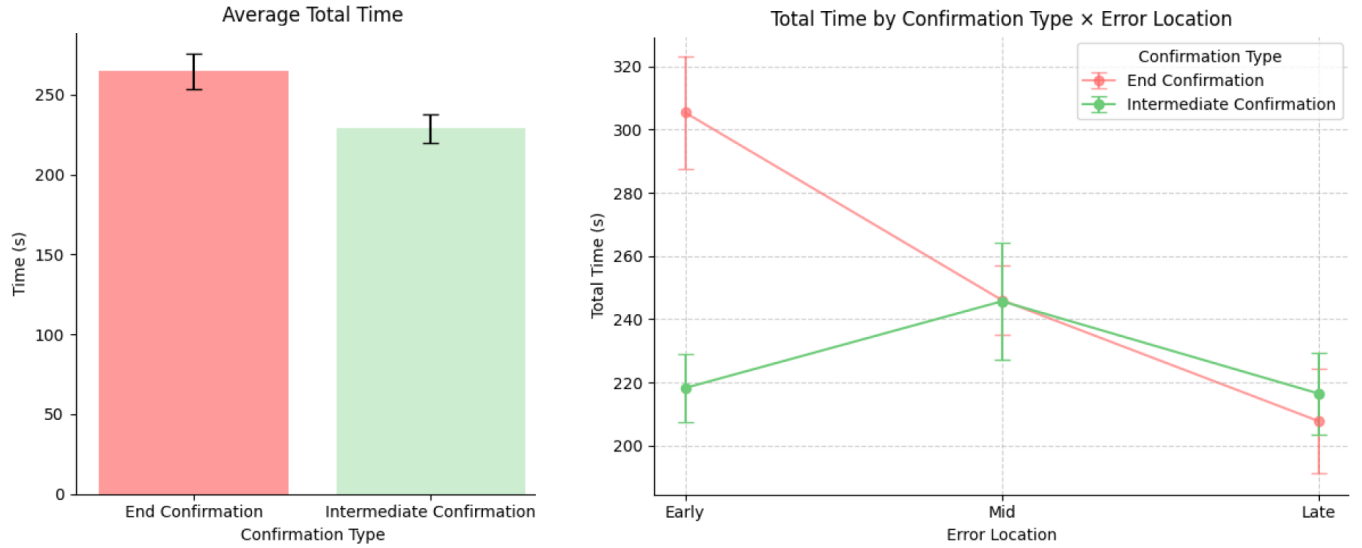


Figure 7: Average completion time under different confirmation strategies. Left: Intermediate confirmation reduced total time compared to end confirmation. Right: Effects vary by error location—large gains for early errors, minimal for mid, and slight cost for late errors. Error bars show 95% CIs.

helped prevent harmful errors (83%). Notably, 77% of participants disagreed that intermediate checkpoints disrupted their task flow. As P18 noted, “I chose intermediate confirmation because it provides a balance between efficiency and reliability. By allowing the system to confirm at selected checkpoints, I can catch potential mistakes earlier instead of waiting until the very end, while still avoiding the constant interruptions of confirming every small step. This makes the process feel smoother but still safe.” Overall, our modeled confirmation frequency achieved the intended goal of balancing user control with agent automation.

7.2.2 Impact of Diagnosis. Although diagnosis time accounted for only a small portion of total completion time in the quantitative study, participants emphasized that it had a strong influence on their experience. When errors were revealed only at the end, users reported feeling disoriented, as they had to review all steps at once without clear guidance: “If I can only confirm at the end, I feel confused about how to deal with the result, because I don’t know which step to check.” (P39) Timely checkpoints minimized the effort needed for diagnosis, making error handling less confusing and less mentally taxing for users.

7.2.3 Confirm at Early Stage. The value of confirmations depends on the stage at which they occur. Based on our quantitative analysis, the largest time savings of intermediate confirmation arose when the agent made mistakes early in the task. The user survey further reinforced this finding from an experiential perspective, highlighting the importance of early-stage confirmation for both efficiency and usability. Each time a user and agent collaborate on a new task, the interaction is a dynamic process of gradually deepening mutual understanding: the agent gains a clearer understanding of the user’s needs, while the user gains a better sense

of the agent’s performance and progressively builds trust in its automatic execution for that specific task. As P9 remarked:

“With an employee I haven’t fully developed trust in yet, I would want to see intermediate steps in their work just to check that they’ve done it right. Perhaps if the employee has shown me that they can do the tasks correctly several times without errors, they can just show me their work at the end. I can’t trust that it’s going to do the task right all the way to the end quite yet, so I want it to show me that it has done the intermediate steps right before getting to a wrong final product.”

This perspective is also reflected in our model. We allow each step to have its own probability p_{a_i} , which can initially be set conservatively so that early-stage confirmations are more frequent. As users catch and correct early errors, p_{a_i} increases, and the model dynamically decreases confirmation frequency. This adaptive behavior was not fully captured in our simulated environment. We will discuss in the next section how such dynamics could be more realistically incorporated.

7.2.4 Subjective Perception. Participants’ preferences for confirmation strategy also reflected their perceptions of task importance and difficulty. For important tasks, they preferred more frequent checks (e.g., “something elaborate like trip planning”), while for trivial tasks (e.g., “adding groceries to my cart”), they preferred confirmation only at the end (P20). Perceived difficulty also mattered: users expected easier tasks to require fewer confirmations. However, user perceptions often diverged from reality. For tasks they considered simple, participants expected fewer confirmations, yet were frustrated when the agent still made errors. For example, agents frequently failed in the shopping cart task, but many users

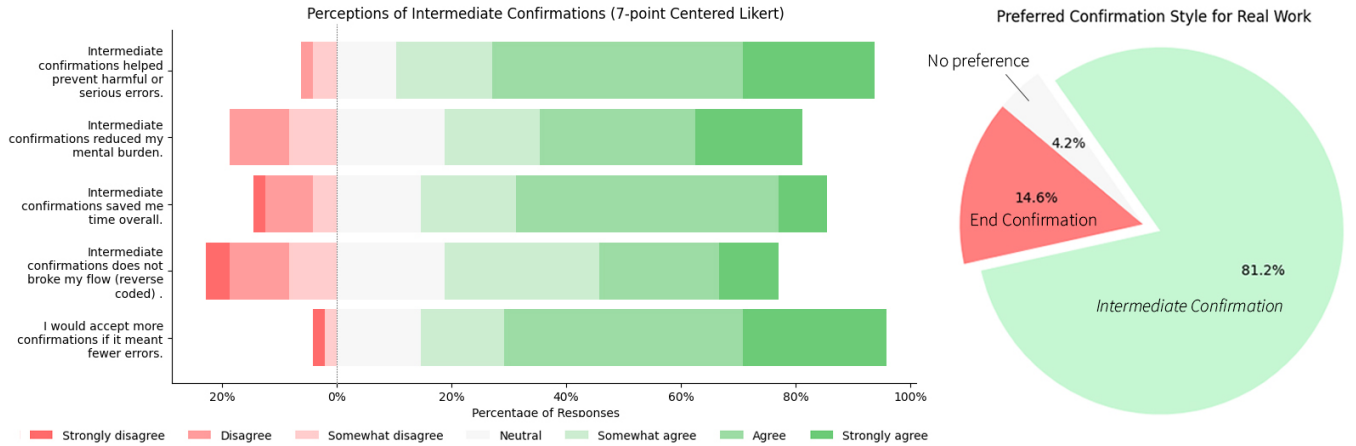


Figure 8: Participant Likert scale scores for the questions asked in our post-study survey (Left). Preferred confirmation style (Right)

saw this task as trivial. Since our model calculates confirmation frequency from actual error rates, it correctly prescribed more frequent checkpoints. While efficient in theory, such frequent confirmations may feel disruptive when they conflict with user expectations. In the next section, we discuss how confirmation strategies can be better aligned with user perceptions in future systems.

8 Discussion

Over the past decades, mixed-initiative systems have been moving from user-led tools [7, 50, 71] toward increasingly agent-led workflows [6, 64]. As Tennenhouse [86] notes, humans shift from being “in the loop” on every step to acting as supervisors and policy-makers. Yet most current agents are still fundamentally reactive: they either wait for users to notice problems and intervene, or they only ask for input when they cannot proceed on their own. Our work aims at the next step in this trajectory: agents that remain mostly autonomous, but proactively allocate supervision by deciding when users should be brought back into the loop. Our scheduler can be plugged into existing systems as a timing layer that determines when to pause, ask for clarification, or surface plans for review. To move toward this vision, we consider two complementary directions: (1) what else to model—expanding beyond efficiency to capture subjective cost, safety, and trust, and (2) how else to design—reimagining confirmation as an interaction paradigm rather than a discrete interruption.

8.1 Beyond Efficiency: Cost, Safety and Trust

Our current model treats time as the only cost factor when determining the best confirmation points. However, in real-world applications, user decisions about when to confirm are influenced by broader considerations. As noted in Section 7.2.4, subjective perceptions play a critical role; here, we expand on this point by examining two key dimensions: real-world cost and user-perceived reliability.

8.1.1 Real-world cost and safety. In practice, agentic failures can have an outsized real-world impact that extends far beyond execution time. Errors may lead to financial loss (e.g., rescheduling fees for a wrong flight), emotional or social consequences (e.g., accidentally sending an email to the wrong recipient), or even irreversible risks (e.g., permanently deleting a file) and harms (e.g., privacy breaches, opening phishing links, manipulative suggestions) [4, 5, 11]. Strategic human oversight is therefore crucial for addressing these safety issues. Since human oversight is expensive, it cannot be applied uniformly to every low-level action; instead, it must be focused where it matters most [5, 33]. Our CDCR framework speaks directly to this question of where oversight should be applied. Future work should expand the range of costs that can be supported. For general actions, the replanning term (e.g., t_{redo}) can be extended to support multi-dimensional cost functions that balance heterogeneous factors, such as time, money, emotional costs, and risks, replacing the purely time-based redo cost used in our current instantiation. Further, future work should also explore the use of mandatory checkpoints for high-stakes actions that occur regardless of the system’s estimated accuracy.

8.1.2 Trust calibration across accuracy regimes. Users’ willingness to confirm is shaped not only by actual system accuracy but also by how reliable they *believe* the agent to be, based on subjective experience and agent framing [47, 66, 79]. Prior work shows that overly positive framings of system performance can inflate perceived reliability and encourage overuse even when this harms task outcomes [18, 73, 74]. Benchmark evaluations can exacerbate this gap: agents may appear highly capable on curated benchmarks while generalizing poorly to practical real-world tasks [30]. In such settings, optimistic benchmark-based messaging can lead users to over-trust the agent and reduce or even cease verification, reinforcing automation complacency. Our model offers a way to counter this by grounding the per-step correctness probabilities p_{a_i} in observational real-world performance data (e.g., deployment logs or field studies) rather than benchmark scores alone. In that case, our

model’s recommended checkpoint schedule will be more conservative, supporting healthier user oversight and making benchmark optimism visible. As agent performance improves and true error rates drop, the role of user verification in our model correspondingly shifts: its value is less about catching frequent errors and more about maintaining calibrated trust, ensuring users retain occasional oversight instead of becoming fully complacent.

8.2 Designing Confirmation for Transparency

To better balance user control with agent automation, confirmation should be treated not only as a scheduling problem but also as an interaction design challenge. We focus on three aspects: goal alignment, information exposure, and integration with downstream agents for transparency.

8.2.1 Plan alignment and control hooks. Our findings on error location highlight the importance of prevention. Errors introduced early often stem from poorly communicated goals rather than faulty execution [11]. While most HCI work emphasizes co-execution or error repair [8, 27, 28], we identify a design opportunity in early-stage goal alignment. Before execution, agents could present a structured plan for user review, drawing on concepts from formal languages that use rigorous specifications to evaluate whether task goals are satisfied [1, 63, 78]. Recent agentic interfaces already move in this direction by exposing intermediate plans and approval guards to users, allowing them to correct goals, such as Magentic-UI and Ongoal [20, 61]. As noted in Section 7.2.3, our model assigns each agent action its own correctness probability p_{a_i} , which can be used to decide which parts of a proposed plan warrant mandatory review or additional clarification. In other words, these downstream agents define the interaction hooks for goal alignment, while our scheduler provides a way to time and target those hooks on the most consequential steps.

8.2.2 Targeted transparency and information granularity. As tasks grow longer, participants in our study found confirmation increasingly difficult: extended linear logs made it hard to locate issues, leaving users disoriented. This highlights a core design question—what level of detail should agents expose for effective supervision? Current UIs that present step-by-step activity lists with screenshots are often too low-level, overwhelming users with detail without offering structure. Recent systems surface the same tension in different domains: some agents selectively pause at ambiguous decision points [28, 38, 69], some agents visualize execution traces in hierarchical way [114], and image agents maintain explicit intent states or reuse prior user preference models to reduce the number of refinement rounds [32, 53]. All of these systems must implicitly decide how much internal state and history to show, and when asking for additional input is worth the user’s effort. Agents should offer *targeted transparency*, surfacing additional detail, explanations, or plan summaries only at checkpoints where the expected benefit of more information outweighs the cost of user attention. Hierarchical representations of agent behavior are one concrete instantiation of this idea: by grouping low-level actions into higher-level chunks, interfaces can let users quickly scan the structure of a run and then drill down only where something looks suspicious. If such hierarchical structuring is adopted, our CDCR pattern remains intact,

but diagnosis time may decrease in a logarithmic fashion—since users could locate errors by checking higher-level chunks first (e.g., binary or ternary splits) rather than scanning all low-level steps. Our scheduler provides a normative basis for deciding where these targeted transparency checkpoints should sit. The same logic can be used to decide when to reveal more of the agent’s internal state as models become more accurate and routine errors become rare, shifting the role of confirmation from catching frequent mistakes to timing when additional transparency is most valuable.

9 Conclusion

This paper introduced a decision-theoretic model for determining when users should confirm agent actions in long-horizon tasks. Building on insights from a formative study that surfaced the recurring CDCR pattern and dissatisfaction with confirm-at-end strategies, we conducted a controlled experiment with 48 participants and found that intermediate confirmation reduced task completion time by 13.54% and was strongly preferred by 81% of participants. Looking ahead, our findings suggest that confirmation should be framed not as a binary trade-off between autonomy and oversight, but as a mixed-initiative design opportunity, pointing to future work on integrating efficiency, trust, and interaction design to create more reliable and user-supervised agentic systems.

References

- [1] Constructions Aeronautiques, Adele Howe, Craig Knoblock, ISI Drew McDermott, Ashwin Ram, Manuela Veloso, Daniel Weld, David Wilkins Sri, Anthony Barrett, Dave Christianson, et al. 1998. Pddl—the planning domain definition language. *Technical Report, Tech. Rep.* (1998).
- [2] Layla AI. 2024. About me, Layla: AI Trip Planner and Travel Sidekick. <https://layla.ai/about>. Accessed: July 2025.
- [3] Pokee AI. 2024. Revolutionize Digital Productivity with AI-Powered Workflow Automation. <https://pokee.ai/home>. Accessed: July 2025.
- [4] Lize Alberts, Ulrik Lyngs, and Max Van Kleek. 2024. Computers as bad social actors: Dark patterns and anti-patterns in interfaces that act socially. *Proceedings of the ACM on Human-Computer Interaction* 8, CSCW1 (2024), 1–25.
- [5] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565* (2016).
- [6] Anthropic. 2024. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku. <https://www.anthropic.com/news/3-5-models-and-computer-use>. Accessed: July 2025.
- [7] Riku Arakawa, Hiromu Yakura, and Mayank Goel. 2024. PrISM-Observer: Intervention agent to help users perform everyday procedures sensed using a smartwatch. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. 1–16.
- [8] Ian Arawjo, Chelse Swoopes, Priyan Vaithilingam, Martin Wattenberg, and Elena L Glassman. 2024. Chainforge: A visual toolkit for prompt engineering and llm hypothesis testing. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–18.
- [9] Zahra Ashktorab, Mohit Jain, Q Vera Liao, and Justin D Weisz. 2019. Resilient chatbots: Repair strategy preferences for conversational breakdowns. In *Proceedings of the 2019 CHI conference on human factors in computing systems*. 1–12.
- [10] Rui Assis and Pedro Carmona Marques. 2021. A dynamic methodology for setting up inspection time intervals in conditional preventive maintenance. *Applied Sciences* 11, 18 (2021), 8715.
- [11] Gagan Bansal, Jennifer Wortman Vaughan, Saleema Amershi, Eric Horvitz, Adam Fournier, Hussein Mozannar, Victor Dibia, and Daniel S Weld. 2024. Challenges in human-agent communication. *arXiv preprint arXiv:2412.10380* (2024).
- [12] Richard Barlow and Larry Hunter. 1960. Optimum preventive maintenance policies. *Operations research* 8, 1 (1960), 90–100.
- [13] Hugh Beyer and Karen Holtzblatt. 1999. Contextual design. *interactions* 6, 1 (1999), 32–42.
- [14] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinhe Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Buckner, et al. 2024. Windows agent arena: Evaluating multi-modal os agents at scale. *arXiv preprint arXiv:2409.08264* (2024).

- [15] Virginia Braun and Victoria Clarke. 2012. *Thematic analysis*. 57–71.
- [16] Matthew Chang, Gunjan Chhablani, Alexander Clegg, Mikael Dallaire Cote, Ruta Desai, Michal Hlavac, Vladimir Karaschuk, Jacob Krantz, Roozbeh Mottaghi, Priyam Parashar, et al. 2024. PARTNR: A Benchmark for Planning and Reasoning in Embodied Multi-agent Tasks. *arXiv preprint arXiv:2411.00081* (2024).
- [17] Weihao Chen, Xiaoyu Liu, Jiacheng Zhang, Ian Long Lam, Zhicheng Huang, Rui Dong, Xinyu Wang, and Tianyi Zhang. 2023. Miwa: Mixed-initiative web automation for better user control and confidence. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 1–15.
- [18] Andy Cockburn, Blaine Lewis, Philip Quinn, and Carl Gutwin. 2020. Framing effects influence interface feature decisions. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. 1–11.
- [19] Andy Cockburn, Philip Quinn, Carl Gutwin, Zhe Chen, and Pang Suwanaposee. 2022. Probability weighting in interactive decisions: Evidence for overuse of bad assistance, underuse of good assistance. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–12.
- [20] Adam J Coscia, Shunan Guo, Eunye Koh, and Alex Endert. 2025. OnGoal: Tracking and Visualizing Conversational Goals in Multi-Turn Dialogue with Large Language Models. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*. 1–18.
- [21] CrewAI. 2024. What is CrewAI? <https://docs.crewai.com/en/introduction>. Accessed: July 2025.
- [22] Anindya Das Antar, Somayeh Molaei, Yan-Ying Chen, Matthew L Lee, and Nikola Banovic. 2024. VIME: Visual Interactive Model Explorer for Identifying Capabilities and Limitations of Machine Learning Models for Sequential Decision-Making. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. 1–21.
- [23] Bram De Jonge and Philip A Scarf. 2020. A review on maintenance optimization. *European journal of operational research* 285, 3 (2020), 805–824.
- [24] Rommert Dekker, Ralph E Wildeman, and Frank A Van der Duyn Schouten. 1997. A review of multi-component maintenance models with economic dependence. *Mathematical methods of operations research* 45, 3 (1997), 411–435.
- [25] Zane Durante, Qiuyuan Huang, Naoki Wake, Ran Gong, Jae Sung Park, Bidipta Sarkar, Rohan Taori, Yusuke Noda, Demetri Terzopoulos, Yejin Choi, et al. 2024. Agent ai: Surveying the horizons of multimodal interaction. *arXiv preprint arXiv:2401.03568* (2024).
- [26] Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Peter West, Chandra Bhagavatula, Roman Le Bras, Jena D Hwang, et al. 2023. Faith and fate: Limits of transformers on compositionality (2023). *arXiv preprint arXiv:2305.18654* (2023).
- [27] Will Epperson, Gagan Bansal, Victor C Dibia, Adam Fourney, Jack Gerrits, Erkang Zhu, and Saleema Amershi. 2025. Interactive debugging and steering of multi-agent ai systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–15.
- [28] KJ Feng, Kevin Pu, Matt Latzke, Tal August, Pao Siangliulue, Jonathan Bragg, Daniel S Weld, Amy X Zhang, and Joseph Chee Chang. 2024. Cocoa: Co-planning and co-execution with ai agents. *arXiv preprint arXiv:2412.10999* (2024).
- [29] Flowith. 2024. Introduction to Flow Mode. <https://doc.flowwith.io/flow-mode-general-mode/introduction-to-flow-mode>. Accessed: July 2025.
- [30] James Fodor. 2025. Line goes up? inherent limitations of benchmarks for evaluating large language models. *arXiv preprint arXiv:2502.14318* (2025).
- [31] Genspark. 2024. Understanding Genspark.AI. <https://genspark.ai/spark/understanding-genspark-ai>. Accessed: July 2025.
- [32] Meera Hahn, Wenjun Zeng, Nithish Kannan, Rich Galt, Kartikeya Badola, Been Kim, and Zi Wang. 2025. Proactive Agents for Multi-Turn Text-to-Image Generation Under Uncertainty. In *Forty-second International Conference on Machine Learning*.
- [33] Andreas Holzinger, Kurt Zatloukal, and Heimo Müller. 2025. Is human oversight to AI systems still possible? 59–62 pages.
- [34] Eric Horvitz. 1999. Principles of mixed-initiative user interfaces. In *Proceedings of the SIGCHI conference on Human Factors in Computing Systems*. 159–166.
- [35] Siyuan Hu, Mingyu Ouyang, Difei Gao, and Mike Zheng Shou. 2024. The dawn of gui agent: A preliminary case study with claude 3.5 computer use. *arXiv preprint arXiv:2411.10323* (2024).
- [36] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. 2024. Understanding the planning of LLM agents: A survey. *arXiv preprint arXiv:2402.02716* (2024).
- [37] Faria Huq, Jeffrey P Bigham, and Nikolas Martelaro. 2023. "What's important here?": Opportunities and Challenges of Using LLMs in Retrieving Information from Web Interfaces. *arXiv preprint arXiv:2312.06147* (2023).
- [38] Faria Huq, Zora Zhiruo Wang, Frank F Xu, Tianyue Ou, Shuyan Zhou, Jeffrey P Bigham, and Graham Neubig. 2025. CowPilot: A Framework for Autonomous and Human-Agent Collaborative Web Navigation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (System Demonstrations)*. 163–172.
- [39] Deniz Iren and Semih Bilgen. 2014. Cost of quality in crowdsourcing. *Human Computation* 1, 2 (2014).
- [40] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *Comput. Surveys* 55, 12 (2023), 1–38.
- [41] Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. 2024. DSbench: How Far Are Data Science Agents from Becoming Data Science Experts? *arXiv preprint arXiv:2409.07703* (2024).
- [42] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221* (2022).
- [43] Sayash Kapoor, Benedikt Stroebel, Zachary S Siegel, Nitya Nadgir, and Arvind Narayanan. 2024. Ai agents that matter. *arXiv preprint arXiv:2407.01502* (2024).
- [44] Majeed Kazemitabaar, Jack Williams, Ian Drosos, Tovi Grossman, Austin Zachary Henley, Carina Negreanu, and Advait Sarkar. 2024. Improving steering and verification in AI-assisted data analysis with interactive task decomposition. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. 1–19.
- [45] Kevin C Lam, Carl Gutwin, Madison Klarkowski, and Andy Cockburn. 2022. More Errors vs. Longer Commands: The Effects of Repetition and Reduced Expressiveness on Input Interpretation Error, Learning, and Effort. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–17.
- [46] Lane Lawley and Christopher J. MacLellan. 2024. VAL: Interactive Task Learning with GPT Dialog Parsing. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*. doi:10.1145/3613904.3641915
- [47] John D Lee and Katrina A See. 2004. Trust in automation: Designing for appropriate reliance. *Human factors* 46, 1 (2004), 50–80.
- [48] Ariel Levy, Monica Agrawal, Arvind Satyanarayan, and David Sontag. 2021. Assessing the impact of automated suggestions on decision making: Domain experts mediate model errors but take less initiative. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 1–13.
- [49] Amanda Li, Jason Wu, and Jeffrey P Bigham. 2023. Using LLMs to Customize the UI of Webpages. In *Adjunct Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 1–3.
- [50] Chenyi Li, Guande Wu, Gromit Yeuk-Yin Chan, Dishita Gdi Turakhia, Sonia Castelo Quispe, Dong Li, Leslie Welch, Claudio Silva, and Jing Qian. 2025. Satori: Towards Proactive AR Assistant with Belief-Desire-Intention User Modeling. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–24.
- [51] Toby Jia-Jun Li, Amos Azaria, and Brad A Myers. 2017. SUGLITE: creating multimodal smartphone automation by demonstration. In *Proceedings of the 2017 CHI conference on human factors in computing systems*. 6038–6049.
- [52] Toby Jia-Jun Li, Jingya Chen, Haijun Xia, Tom M Mitchell, and Brad A Myers. 2020. Multi-modal repairs of conversational breakdowns in task-oriented dialogs. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*. 1094–1107.
- [53] Zhipeng Li, Yi-Chi Liao, and Christian Holz. 2025. Efficient Visual Appearance Optimization by Learning from Prior Preferences. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*. 1–23.
- [54] Henry Lieberman. 1997. Autonomous interface agents. In *Proceedings of the ACM SIGCHI Conference on Human factors in computing systems*. 67–74.
- [55] Jijia Liu, Chao Yu, Jiaxuan Gao, Yuqing Xie, Qingmin Liao, Yi Wu, and Yu Wang. 2023. Llm-powered hierarchical language agent for real-time human-ai coordination. *arXiv preprint arXiv:2312.15224* (2023).
- [56] Alexandra Sasha Luccioni, Sylvain Viguier, and Anne-Laure Ligozat. 2023. Estimating the carbon footprint of bloom, a 176b parameter language model. *Journal of machine learning research* 24, 253 (2023), 1–15.
- [57] Reuben Luera, Ryan A Rossi, Alexa Siu, Franck Deroncourt, Tong Yu, Sungchul Kim, Ruiyi Zhang, Xiang Chen, Hanieh Salehy, Jian Zhao, et al. 2024. Survey of User Interface Design and Interaction Techniques in Generative AI Applications. *arXiv preprint arXiv:2410.22370* (2024), 1–42.
- [58] Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. 2024. Spreadsheetbench: Towards challenging real world spreadsheet manipulation. *Advances in Neural Information Processing Systems* 37 (2024), 94871–94908.
- [59] Pattie Maes. 1995. Agents that reduce work and information overload. In *Readings in human-computer interaction*. Elsevier, 811–821.
- [60] Manus. 2024. Leave it to Manus: General AI Agent for Work and Life. <https://manus.im/home>. Accessed: July 2025.
- [61] Hussein Mozannar, Gagan Bansal, Cheng Tan, Adam Fourney, Victor Dibia, Jingya Chen, Jack Gerrits, Tyler Payne, Mathews Kunzler Maldaner, Madeleine Grunde-McLaughlin, et al. 2025. Magentic-ui: Towards human-in-the-loop agentic systems. *arXiv preprint arXiv:2507.22358* (2025).
- [62] Magnus Müller and Gregor Žunič. 2024. *Browser Use: Enable AI to control your browser*. <https://github.com/browser-use/browser-use>
- [63] Dana S Nau, Tsz-Chiu Au, Oktay Ilghami, Ugur Kuter, J William Murdock, Dan Wu, and Fusun Yaman. 2003. SHOP2: An HTN planning system. *Journal*

- of artificial intelligence research 20 (2003), 379–404.
- [64] OpenAI. 2025. Introducing ChatGPT agent: bridging research and action. <https://openai.com/index/introducing-chatgpt-agent>. Accessed: July 2025.
 - [65] Vishal Pallagani, Bharath Chandra Muppasani, Kaushik Roy, Francesco Fabiano, Andrea Loreggia, Keerthiram Murugesan, Biplav Srivastava, Francesca Rossi, Lior Hoshen, and Amit Sheth. 2024. On the prospects of incorporating large language models (llms) in automated planning and scheduling (aps). In *Proceedings of the International Conference on Automated Planning and Scheduling*, Vol. 34. 432–444.
 - [66] Raja Parasuraman and Dietrich H Manzey. 2010. Complacency and bias in human use of automation: An attentional integration. *Human factors* 52, 3 (2010), 381–410.
 - [67] David Patterson, Joseph Gonzalez, Urs Hölzle, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David R So, Maud Texier, and Jeff Dean. 2022. The carbon footprint of machine learning training will plateau, then shrink. *Computer* 55, 7 (2022), 18–28.
 - [68] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. 2021. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350* (2021).
 - [69] Yi-Hao Peng, Dingzeyu Li, Jeffrey P Bigham, and Amy Pavel. 2025. Morae: Proactively pausing ui agents for user choices. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*. 1–14.
 - [70] Kevin Pu, Daniel Lazaro, Ian Arawjo, Haijun Xia, Ziang Xiao, Toví Grossman, and Yan Chen. 2025. Assistance or disruption? exploring and evaluating the design and trade-offs of proactive ai programming support. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–21.
 - [71] Kevin Pu, Ting Zhang, Naveen Senthilnathan, Sebastian Freitag, Raj Sodhi, and Tanya R Jonker. 2025. ProMemAssist: Exploring Timely Proactive Assistance Through Working Memory Modeling in Multi-Modal Wearable Devices. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*. 1–19.
 - [72] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. 2025. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326* (2025).
 - [73] Philip Quinn and Andy Cockburn. 2016. When bad feels good: Assistance failures and interface preferences. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. 4005–4010.
 - [74] Philip Quinn and Andy Cockburn. 2020. Loss aversion and preferences in interaction. *Human-Computer Interaction* 35, 2 (2020), 143–190.
 - [75] rabbit research team. 2023. Learning human actions on computer applications. <https://rabbit.tech/research>
 - [76] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.
 - [77] Yijia Shao, Vinay Samuel, Yucheng Jiang, John Yang, and Diyi Yang. 2024. Collaborative gym: A framework for enabling and evaluating human-agent collaboration. *arXiv preprint arXiv:2412.15701* (2024).
 - [78] Vikas Shivashankar, Ron Alford, Ugur Kuter, and Dana S Nau. 2013. Hierarchical goal networks and goal-driven autonomy: Going where ai planning meets goal reasoning. In *Goal Reasoning: Papers from the ACS Workshop*, Vol. 95.
 - [79] Ben Shneiderman. 2020. Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human-Computer Interaction* 36, 6 (2020), 495–504.
 - [80] Ben Shneiderman and Pattie Maes. 1997. Direct manipulation vs. interface agents. *interactions* 4, 6 (1997), 42–61.
 - [81] Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. 2024. Chain of thoughtlessness? an analysis of cot in planning. *Advances in Neural Information Processing Systems* 37 (2024), 29106–29141.
 - [82] Sangho Suh, Meng Chen, Bryan Min, Toby Jia-Jun Li, and Haijun Xia. 2024. Luminate: Structured generation and exploration of design space with large language models for human-ai co-creation. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–26.
 - [83] Theodore Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas Griffiths. 2023. Cognitive architectures for language agents. *Transactions on Machine Learning Research* (2023).
 - [84] Sharareh Taghipour and Dragan Banjevic. 2012. Optimal inspection of a complex system subject to periodic and opportunistic inspections and preventive replacements. *European Journal of Operational Research* 220, 3 (2012), 649–660.
 - [85] Mike Ten Wolde and Adel A Ghobbar. 2013. Optimizing inspection intervals—Reliability and availability in terms of a cost model: A case study on railway carriers. *Reliability Engineering & System Safety* 114 (2013), 137–147.
 - [86] David Tenenhouse. 2000. Proactive computing. *Commun. ACM* 43, 5 (2000), 43–50.
 - [87] Sebastian Thrun. 2002. Probabilistic robotics. *Commun. ACM* 45, 3 (2002), 52–57.
 - [88] Kashyap Todi, Gilles Bailly, Luis Leiva, and Antti Oulasvirta. 2021. Adapting user interfaces with model-based reinforcement learning. In *Proceedings of the 2021 CHI conference on human factors in computing systems*. 1–13.
 - [89] Browser Use. 2024. The AI browser agent. <https://browser-use.com>. Accessed: July 2025.
 - [90] Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. 2023. PlanBench: An extensible benchmark for evaluating large language models on planning and reasoning about change. *Advances in Neural Information Processing Systems* 36 (2023), 38975–38987.
 - [91] Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. 2024. On the planning abilities of large language models—a critical investigation. *Advances in Neural Information Processing Systems* 36 (2024).
 - [92] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv:2305.16291* [cs.AI]
 - [93] Hongzhou Wang. 2002. A survey of maintenance policies of deteriorating systems. *European journal of operational research* 139, 3 (2002), 469–489.
 - [94] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.
 - [95] Wenbin Wang. 2012. An overview of the recent advances in delay-time-based maintenance modelling. *Reliability Engineering & System Safety* 106 (2012), 165–178.
 - [96] Xihuai Wang, Shao Zhang, Wenhao Zhang, Wentao Dong, Jingxiao Chen, Ying Wen, and Weinan Zhang. 2024. Zsc-eval: An evaluation toolkit and benchmark for multi-agent zero-shot coordination. *Advances in Neural Information Processing Systems* 37 (2024), 47344–47377.
 - [97] Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Ma, and Yitao Liang. 2023. Describe, Explain, Plan and Select: Interactive Planning with Large Language Models Enables Open-World Multi-Task Agents. *arXiv:2302.01560* [cs.AI]
 - [98] Nicholas R Waytowich, Devin White, MD Sunbeam, and Vinicius G Goecks. 2024. Atari-gpt: Investigating the capabilities of multimodal large language models as low-level policies for atari games. *arXiv e-prints* (2024), arXiv–2408.
 - [99] Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. 2025. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516* (2025).
 - [100] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
 - [101] Justin D Weisz, Jessica He, Michael Muller, Gabriela Hoefler, Rachel Miles, and Werner Geyer. 2024. Design principles for generative AI applications. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–22.
 - [102] Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanguang Xiao, and Yu Su. 2024. Travelplanner: A benchmark for real-world planning with language agents. *arXiv preprint arXiv:2402.01622* (2024).
 - [103] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. 2024. Oworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems* 37 (2024), 52040–52094.
 - [104] Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. 2024. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. *arXiv:2412.14161* [cs.CL] <https://arxiv.org/abs/2412.14161>
 - [105] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems* 35 (2022), 20744–20757.
 - [106] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.
 - [107] Enhao Zhang, Erkang Zhu, Gagan Bansal, Adam Fournay, Hussein Mozannar, and Jack Gerrits. 2025. Optimizing sequential multi-step tasks with parallel llm agents. *arXiv preprint arXiv:2507.08944* (2025).
 - [108] Shao Zhang, Xihuai Wang, Wenhao Zhang, Chaoran Li, Junru Song, Tingyu Li, Lin Qiu, Xuezhi Cao, Xunliang Cai, Wen Yao, et al. 2025. Leveraging dual process theory in language agent framework for real-time simultaneous human-AI collaboration. *arXiv preprint arXiv:2502.11882* (2025).
 - [109] Hang Zhao, Sophia Gu, Chun Yu, and Xiaojun Bi. 2022. Bayesian hierarchical pointing models. In *Proceedings of the 35th annual acm symposium on user interface software and technology*. 1–13.

- [110] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. 2024. Retrieval-augmented generation for ai-generated content: A survey. *arXiv preprint arXiv:2402.19473* (2024).
- [111] Yuheng Zhao, Junjie Wang, Linbin Xiang, Xiaowen Zhang, Zifei Guo, Cagatay Turkay, Yu Zhang, and Siming Chen. 2024. Lightva: Lightweight visual analytics with llm agent-based task planning and execution. *IEEE Transactions on Visualization and Computer Graphics* (2024).
- [112] Qingxiao Zheng, Yiliu Tang, Yiren Liu, Weizi Liu, and Yun Huang. 2022. UX research on conversational human-AI interaction: A literature review of the ACM digital library. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–24.
- [113] Jieyu Zhou. 2025. OrchVis: Hierarchical Multi-Agent Orchestration for Human Oversight. *arXiv preprint arXiv:2510.24937* (2025).
- [114] Jieyu Zhou and Christopher MacLellan. 2024. Improving Interface Design in Interactive Task Learning for Hierarchical Tasks based on a Qualitative Study. In *Adjunct Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. 1–3.
- [115] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. 2023. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854* (2023).
- [116] Xiaojun Zhou, Kaimin Huang, Lifeng Xi, and Jay Lee. 2015. Preventive maintenance modeling for multi-component systems with considering stochastic failures and disassembly sequence. *Reliability Engineering & System Safety* 142 (2015), 231–237.

A Formative Study Task Prompt

A.1 Task 1

Fill the cart on Kroger.com for a grocery delivery at 30332. I need all the ingredients to make a pizza at home. Maximize the quality of the ingredients while staying under the budget of 20 dollars.

Observed Errors: The agent mentioned adding certain items to the cart but those items were not present at the end of the run. The final summary from the model did not match the contents of the cart.

A.2 Task 2

Please help me edit this image. I want to create an advertisement for Coca Cola. Remove all the objects on the table except for the sprite. Change the sprite to coke. Remove the man in the background and change the background to an outdoor scene.

Observed Errors: The agent often applied wrong filters, selected objects improperly. The final result was very far from the desired result.

A.3 Task 3

Help me book a hotel in NYC for July 4 through July 12. It should be in Manhattan, clean, and within walking distance to several cafes. Look for good deals where the hotel is either less expensive than similar quality or on discount from its usual price.

Observed Errors: The agent often selected the wrong dates for flight/hotel bookings.

A.4 Task 4

Please turn these invoices into a spreadsheet which record each expense's time and cost. And then send this spreadsheet to my email.

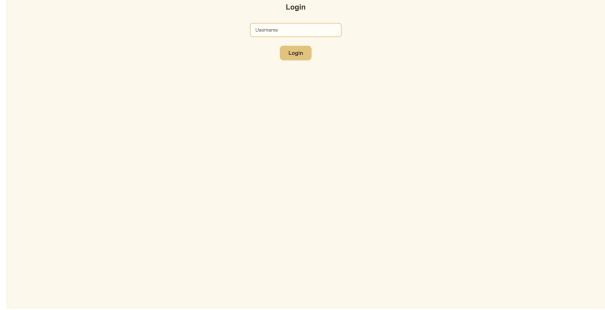
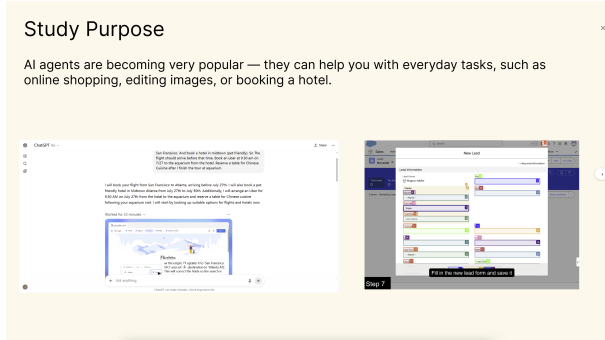
Observed Errors: There was a mismatch between the values provided in the invoice and the ones entered into the spreadsheet.

A.5 Task 5

Please generate a step-by-step plan for cooking onion soup In the Overcooked game, there are two primitive actions: 1. go to: "goes to and faces the target object, where object is something like pot, onion, etc." 2. interact: "interact with the object, e.g., this should be called if you are trying to interact with the pot, onion dispenser, plate dispenser, tomato dispenser, tomato, onion, etc." 3. wait20: waits for 20 time steps The objects in the environment are pot, onion dispenser, plate dispenser, tomato dispenser, tomato, onion. Rules: 1. you should have onion soup before you cook an onion 2. every time you cook something, you have to wait 20 minutes

Observed Errors: The agent selected the wrong location or did not execute the action it planned.

Demographic Information	Participant Counts
AI Tool Usage Frequency	Never (1%), Occasionally (13%), Weekly (22%), Daily (65%)
Age	18–25 (69%), 26–35 (30%), >55 (1%)
Gender	Male (64%), Female (36%)
AI Tool Usage Proficiency	Unfamiliar (16%), Beginner (18%), Intermediate (39%), Advanced (28%)

Table 1: Participant Information**(a) The Login Screen****(b) The Tutorial Screen.****Figure 9: User Interface Introduction**

B Model Verification Study

B.1 Demographic Data

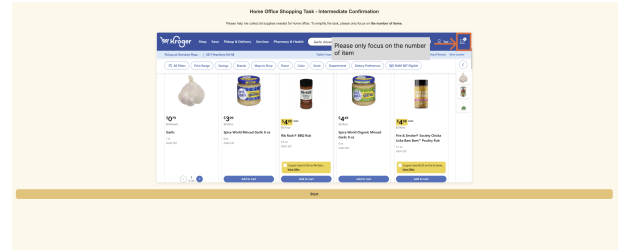
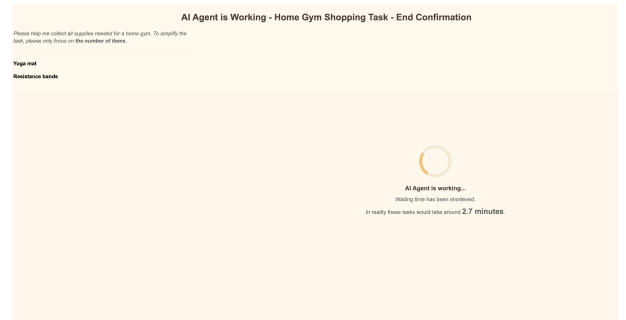
B.2 User Study Design

The user studies were conducted using a custom web application that presented participants with a simulated interface to an AI agent. The application was developed using standard HTML and CSS, with all experiment data logged to a Firebase Firestore database for subsequent analysis. The simulation comprised multiple screens that the user navigated sequentially to progress through the task.

Tutorial and Login : At the onset of the study, each user was presented with a short video tutorial within a modal popup, explaining the system’s UI components. The tutorial demonstrated a person using the identical interface to monitor a simulated agent during a task. The introduction also included annotated images

highlighting key UI elements, such as the agent state display and control buttons. After closing the tutorial, users were asked to enter their participant ID, which was assigned during recruitment.

Task Execution: Upon logging in, users were presented with a task summary and a description of the expected outcome. Data logging was initiated when the user clicked the “Start Task” button, transitioning them to the “Working” page. This page displayed a loading spinner and the text “Agent is working” to simulate the processing latency of a real AI agent between confirmation points.

**(a) Task Start Screen****(b) Working Screen****Figure 10: Main task interface**

Experiment Design : Each participant ID was assigned a fixed sequence of experiments selected from a pool of image and tag pairs outlining the task flow. Each experiment was stored as a JSON object containing the step name, image reference, and a boolean value indicating correctness (used to determine which step contained an error). This sequence was structured to support the study’s within-subjects design.

B.3 User Study Task Description

Task 1: Shopping Cart Task : In this task, the user was presented with a simulated AI agent tasked with purchasing a predetermined list of items from a home delivery platform. The user monitored the agent’s progress and was asked to identify any errors the agent committed whenever a checkpoint was reached.

- **Subtasks:**

- (1) Type the name of the item in the search bar and initiate the search.
- (2) Scroll through the results to identify the cheapest offering.
- (3) Add the selected offering to the cart.

Task 2: Image Editing Task : In this task, the user monitored an AI agent responsible for editing an image to achieve a specific target output. The input images contained several objects that required removal or replacement to generate the final result.

- **Subtasks:**

- (1) Locate the object of interest within the frame.
- (2) Outline the object with high precision.
- (3) Remove or replace the object.
- (4) Repeat the steps above until the task is complete.

Task 3: Overcooked Task : This task involved monitoring an AI agent controlling a character within a simulated version of the game *Overcooked*. The agent was tasked with adding a specific list of ingredients to a pot and periodically adjusting the temperature to correctly cook the dish.

- **Subtasks:**

- (1) Navigate to the item dispenser.
- (2) Retrieve the correct quantity of the specified item.
- (3) Add the item to the pot and adjust the temperature.
- (4) Repeat the previous steps for all listed ingredients.

C Simulation

C.0.1 Setup and Parameterization. We instantiated the image-editing task using the same base parameters as in our main simulation: a 16-step task with $p = 0.91$, $t_{\text{confirm}} = 10$, $t_{\text{diagnose}} = 3$, and $t_{\text{redo}} = 10$. To demonstrate that our model naturally supports heterogeneous, per-step specifications, we introduced controlled step-wise variations on top of this base regime.

C.1 Heterogeneous Step-Wise Parameters

C.1.1 Step-Wise Probabilities and an Error Cluster. We assigned non-uniform correctness probabilities across the 16 steps: the first five steps are moderately reliable ($p_{a_1} = \dots = p_{a_5} = 0.9$), the next four steps form a brittle error-prone cluster ($p_{a_6} = \dots = p_{a_9} = 0.75$), and the remaining seven steps are highly reliable ($p_{a_{10}} = \dots = p_{a_{16}} = 0.95$). As expected, the optimal checkpoint schedule increases its confirmation density within the brittle segment, indicating that the model adapts intervention frequency to localized spikes in error likelihood, as shown in Figure 11.

C.1.2 Step-Wise Timing Parameters. To probe how heterogeneous timing affects the optimal schedule, we next made the confirmation time depend on the position of the step in the task. Specifically, for a 16-step task we set

$$t_{\text{confirm}}^k = 5 + 0.5(k - 1), \quad k = 1, \dots, 16,$$

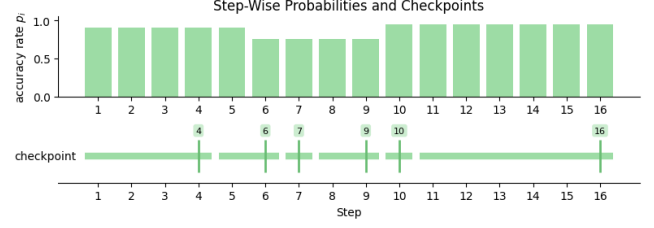


Figure 11: Step-wise correctness probabilities and the corresponding optimal checkpoint schedule

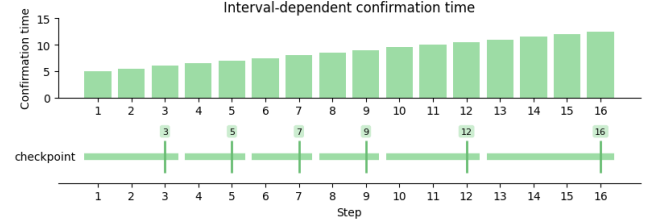


Figure 12: Interval-dependent confirmation time and the resulting optimal checkpoint schedule

so that confirming later steps becomes progressively more expensive. As a result, the optimal policy spreads checkpoints farther apart toward the end of the trajectory, trading off higher accumulated error risk against the rising marginal cost of confirmation, as shown in Figure 12. We additionally simulated step-wise variants of diagnosis time t_{diagnose}^k and execution/redo time t_{redo}^k . Across these heterogeneous timing regimes, we observed consistent qualitative trends: increasing confirmation time leads to wider spacing between checkpoints, while decreasing diagnosis or execution time makes it cheaper to discover and repair errors after they occur, and thus also increases the optimal span between checkpoints:

$$t_{\text{confirm}} \uparrow \Rightarrow \text{span} \uparrow \quad t_{\text{diagnose}} \downarrow \Rightarrow \text{span} \uparrow \quad t_{\text{redo}} \downarrow \Rightarrow \text{span} \uparrow$$

C.2 Sensitivity to Per-Step Reliability

We conducted a simple sensitivity analysis by sweeping the per-step correctness rate p from 0.60 to 0.99 and computing the corresponding optimal checkpoint frequency. As shown in Fig. 13, in this 16-step task, the number of confirmation points decreases sharply as reliability increases. When $p \leq 0.70$, the model places checkpoints at nearly every step, as the compounding error risk quickly overwhelms the cost of confirmation. Once accuracy exceeds $p = 0.80$, checkpoint frequency begins to drop rapidly, and at near-perfect accuracy ($p = 0.99$), the schedule collapses to a single confirmation at the end.

D Simulation on Probability-Based Baselines

As a complement to our user study, we ran a stochastic simulation that instantiates a family of probability-based confirmation policies and compares them against our model’s checkpoint schedule. In all three domains (shopping, image editing, and Overcooked), we reused exactly the same parameters as in the user study, including

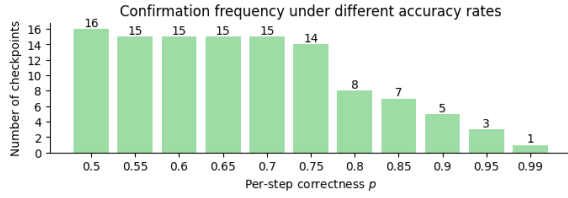


Figure 13: Sensitivity of the optimal checkpoint schedule to uniform per-step correctness in the image-editing domain.

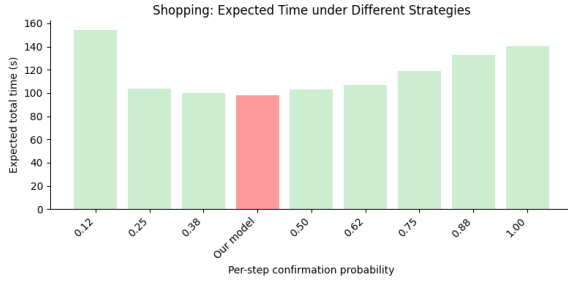


Figure 14: Shopping: Estimated completion time under probability-based confirmation strategies (green) versus our model's scheduler (red).

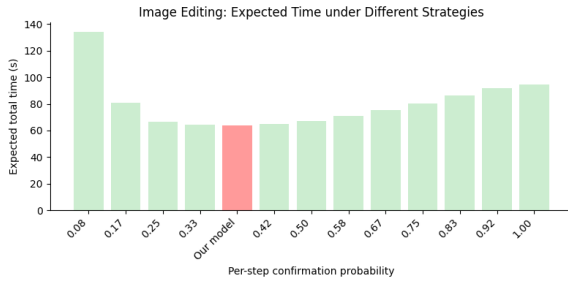


Figure 15: Image-editing: Estimated completion time under probability-based confirmation strategies (green) versus our model's scheduler (red).

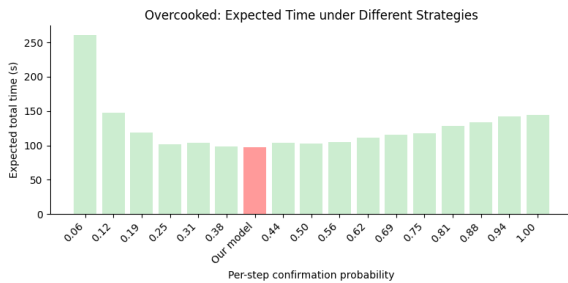


Figure 16: Overcooked: Estimated completion time under probability-based confirmation strategies (green) versus our model's scheduler (red).

task length, per-step action accuracy, and the timing parameters for confirmation, diagnosis, and redo.

Under the probability-based policy, each step has an independent probability p_{check} of becoming a checkpoint. For an N -step task, this yields an expected number of checkpoints of approximately $p_{\text{check}} * N$. To cover a wide range of behaviors, we swept p_{check} over values whose expected number of checkpoints ranges from roughly 1 to N , thus spanning almost all reasonable confirmation frequencies.

For each schedule, we simulated 5,000 random runs. In each run, every step independently succeeded or failed according to the per-step accuracy, and we applied the CDCR pattern to compute the total completion time based on when checkpoints occurred and where the first error appeared. We then averaged the completion times over these 5,000 runs to obtain an empirical estimate of the expected time for that schedule.

Figures 14–16 plot the estimated completion time for all probability-based baselines, with an additional bar for our model. Across all three domains, the probability-based policies exhibit the expected U-shaped pattern: very small p_{check} values lead to few checkpoints and large rollback costs, while very large p_{check} values approach confirming at every step and incur high confirmation overhead. Our model's schedule (red bar) consistently lies the bottom of this curve and slightly lower than, the minimum across all probability baselines in each domain. Because our model uses the same number of checkpoints as the best-performing probability baseline, but optimizes their exact placement rather than only fixing the total count.